

불균형 데이터 구조를 가진 산탄데르
은행 고객만족 분석 실습에서 불만족을
유발하는 요인을 분석하여, 다양한
모델 앙상블과 교차검증을 통해 불만족
고객을 조기 식별하고, 고객 유지
전략의 근거를 확보했다.

제 2 장

산탄데르 은행 고객 만족 평가

제 2 장 목차

2. 산탄데르 은행 고객 만족평가	3
2.1 문제 정의 및 데이터 이해	3
2.1.1 문제 정의	3
2.1.2 데이터 설명	7
2.2 탐색적 데이터 분석 (EDA)	17
2.2.1 통계적 요약	17
2.2.2 시각화 분석	19
2.2.3 발견된 문제점	21
2.3 데이터 전처리	26
2.3.1 결측치 처리	26
2.3.2 Zero 값의 의미 파악 및 처리	27
2.3.2 상관관계 기반 피처 제거	27
2.3.3 이상치 처리	28
2.3.4 로그 변환을 통한 고왜도 보정	29
2.3.5 인코딩	29
2.3.6 스케일링	29
2.3.8 최종 Feature 요약	29
2.4 모델링	31
2.4.1 학습/검증 데이터 분할	31
2.4.2 베이스라인 모델	32
2.4.3 모델 선정	34
2.4.4 하이퍼파라미터 튜닝	35
2.4.5 모델 성능 비교	40
2.5 모델 평가	42

2.5.1	평가지표.....	42
2.5.2	Feature Importance	46
2.5.3	예측 결과 분석	48
2.5.4	모델 해석	51
2.5.4.2		53
2.6	결론 및 시사점	57
2.6.1	결론	57
2.6.2	최종결론 정리	58
2.6.3	시사점과 한계점 및 제언	58
2.7	부록	60
2.7.1	코드 저장소	60
2.7.2	실행 환경	60

2. 산탄데르 은행 고객 만족평가

2.1 문제 정의 및 데이터 이해

2.1.1 문제 정의

2.1.1.1 프로젝트 배경 및 목표

- 프로젝트에서의 산탄데르는 ‘고객 만족’을 전략적 핵심 지표로 삼고 있다.
- **NPS(Net Promoter Score)** 등 다양한 도구로 고객 경험을 측정하고 있으며, Kaggle 데이터셋을 활용한 머신러닝 분석에서 고객 불만족을 예측하는 모델링이 활발히 연구되었다.

1) 산탄데르 은행 고객만족 전략 수립 배경

- **Think Customer 원칙** : 고객을 모든 전략의 중심에 두고, “디지털 은행 + 지점” 모델로 전환
- 측정 방식 :
 - NPS(Net Promoter Score) : **고객이 은행을 추천할 의향**을 수치화
 - 고객 경험 지표: 서비스 품질, 디지털 채널 편의성, 지점 서비스 등 종합적 평가

2) 분석 인사이트

- **불만족 고객 소수의 비율** : 은행 입장에서는 불만족 고객을 정확히 찾아내는 것이 핵심
- **데이터 기반 접근** : 머신러닝을 통해 어떤 특성이 불만족을 유발하는지 파악
- **전략적 활용** :
 - 불만족 고객을 조기 식별 ⇒ 맞춤형 서비스 제공
 - 디지털 채널 개선 ⇒ 고객 경험 향상
 - 지점 서비스와 온라인 서비스의 균형 강화

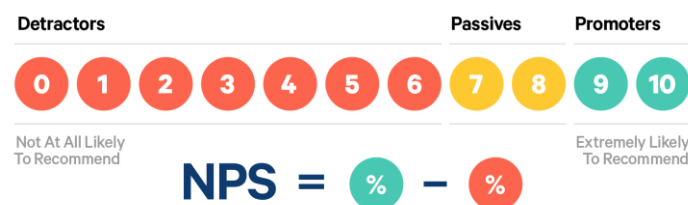
3) 문제 정의 및 핵심 목표

- 산탄데르의 고객 만족 분석은 전략적 지표(NPS)*와 데이터 기반 머신러닝 예측을 결합하여, 고객 충성도와 불만족 관리에 집중

- 글로벌 은행으로서 경쟁력 유지를 위한 핵심 요소
 - NPS 지표 가이드라인과 당시 Kaggle Santander Customer Satisfaction 데이터셋을 분석하여 문제 정의
- 이번 실습 Semi - Project 에서의 산탄데르 고객 만족 분석을 통해 불만족스러운 고객을 초기에 파악하여, **고객 만족도를 개선**하기 위함을 핵심목표로 두고 있음
- 또한, 고객의 프로필, 관심사, 요구사항을 기반으로 맞춤형 상품과 서비스를 제공하고, 각 채널에서 고객과의 관계 만족도를 측정하여 신뢰관계 구축 및 **고객경험(CX) 개선**
- 궁극적으로 고객 충성도를 높이고, 이탈을 방지하며, 비즈니스 성과 개선

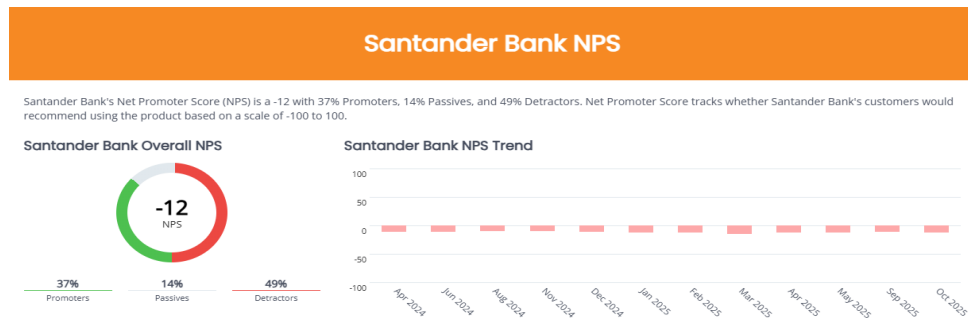
2.1.1.2 NPS(Net Promoter Score)

- NPS 는 고객이 해당 브랜드를 **추천할 의향**이 있는지를 측정하는 지표
- **계산 방식** : 추천 고객(9 10 점) 비율 - 비추천 고객(0 6 점) 비율
- 금융업계에서 **고객 충성도와 장기적 수익성**을 예측하는데 활용
 - 단순 만족도 조사보다 행동 의향(추천/비추천)을 반영하기 때문에 전략적 가치가 크다.
 - 산탄데르 은행의 NPS 를 활용으로 '고객 중심 경영'을 강조해왔다.



[그림 2.1.1 산탄데르 은행 월별 NPS 평가 지표(24' - 25')]

- 2025 년 10 월 기준 산탄데르 은행 NPS 지표는 -12 점으로 추천 고객 37%, 비추천 고객 49%를 차지하였다.
 - NPS 는 산탄데르 은행 고객이 해당 상품을 추천할지 여부를 -100 점에서 100 점 까지의 척도로 평가를 진행한다.



[그림 2.1.2 산탄데르 은행 월별 NPS 평가 지표(24' - 25')]

- 다른 경쟁사 은행과 비교하여 산탄데르 은행은 NPS 4 위를 차지하였고, Td Bank, Citizens National Bank 이 각각 1 위와 2 위를, Bank of America 가 3 위를 기록하였다.
 - 경쟁사 은행들 중 산탄데르 은행이 가장 낮은 기업 가치를 보였다.

How Other Brands Compare

Santander Bank is ranked #4 for NPS among its competitors. Td Bank and Citizens National Bank come in first and second, with Bank of America coming in at third. Among those competitors, it is the lowest valued company behind Bank of America.

	 Santander Bank	 Bank of America	 Td Bank	 Citizens National Bank
Global Ranking	#_	#152	#_	#_
NPS	-12	2	15	10
Social Sentiment 	-	Neutral	-	-
Valuation 	\$59.30B	\$298.80B	\$121.22B	-

[그림 2.1.3 주요 경쟁 은행별 NPS 성과 지표 비교 테이블]

2.1.1.3 현황 분석 및 전략 설정 배경

1) 2016 년 업계 현황 및 배경

- 글로벌 금융위기 이후 회복기
 - 2008 년 금융위기 이후 은행들은 신뢰 회복과 건전성 강화에 집중
- 디지털 बैं킹 확산
 - 모바일 및 온라인 금융 서비스가 급격히 성장하며 고객 기대 수준 변화
- 고객 경험 경쟁 심화
 - 단순 금리, 상품 경쟁에서 벗어나 고객 만족도와 충성도가 핵심 경쟁력으로 부상
- 규제 강화

- 유럽, 미국에서 은행 건전성 규제가 강화되며, 효율성과 투명성 주요과제로 부상

2) 산탄데르 은행의 당시 전략

- 고객 중심 경영
- 충성 고객 확대
- 디지털 전환 가속화
- 글로벌 다각화
- 지속가능성 및 사회적 책임

3) Kaggle 고객 만족 분석 대회

- Kaggle 의 산탄데르 고객 만족 분석 대회(2016) 역시 이와같은 맥락에서 등장했다.
- 고객 데이터를 활용해 **만족도와 충성도를 예측**하고, **NPS 개선**을 위한 인사이트를 얻고자 한다.
- 또한, 당시 디지털 고객 확대와 맞물려, 데이터 기반 의사결정이 필수적이었다.
- 따라서, 대회의 문제 정의는 “산탄데르가 고객 경험을 혁신하고 NPS 를 개선하기 위해, **어떤 요인이 만족도에 영향을 주는지** 데이터 분석을 통해 규명한다” 로 이어진다.

2.1.1.4 고객만족 분석 전략과 기대효과

- 당시 슬로건 : “Simple, Personal, Fair” → 고객 중심 경영 강조
- 충성 고객 확대(1 억 2,500 만 명) 와 디지털 고객 증가(2,100 만 명)는 NPS 개선 목표와 직결
- 디지털 전환은 고객 접점 확대 → NPS 점수 향상에 중요한 요인
- 즉, NPS 는 전략 실행 성과를 측정하는 바로미터 역할을 했음
 - Kaggle 의 고객 만족 분석 대회를 통해 단순히 “만족/불만족”을 예측하는 것이 아니라, NPS 개선을 위한 데이터 기반 인사이트 발굴과 연결할 수 있다.
 - 고객 특성, 서비스 이용 패턴, 불만 요인을 분석 → 추천 고객을 늘리고 비추천 고객을 줄이는 전략 수립 가능하다.
 - 따라서, 대회 목적 및 문제 정의는 곧 NPS 를 높이는 실질적 데이터 분석 과제로 해석할 수 있다.

2.1.2 데이터 설명

2.1.2.1 데이터셋 개요

- 출처 : Banco Santander (스페인 최대 은행)
- 컬럼수 : 371 개 (ID + 369 개 특성 + TARGET)
- 언어 : 스페인어 기반 약어 사용
- 익명화 : 모든 변수명이 개인정보 보호를 위해 암호화 되어 있음
- 식별자 및 최상위 주요 변수
 - ID : 고객 고유 식별 번호 (Primary Key, 순차 증가)
 - var3 : 고객 나이 또는 계좌 개설 후 경과 개월수 (-99999 는 명시적 결측치)
 - var15 : 고객 연령대 / 라이프스타일 코드

[표 2.1.1 산탄데르 고객 만족 예측 데이터셋 변수별 의미]

변 수	의 미
금액(imp)	- 거래 금액 정보 [importe = 금액]
지표(ind)	- 거래 금액 정보 [importe = 금액]
지표(ind)	- 이진 변수 (0 또는 1) [indicador = 지표] - '_0' 접미사는 "미보유/미이용" 상태를 나타냄 (대부분 상호 배타적)
수량(num)	- 횟수/개수 정보
잔액(saldo)	- 계좌 잔액 정보
특수(var36)	- 고객 상태 코드 (0,1,2,3,99 - 99 는 특수상태)
변화량(delta)	- 1 년 3 개월(15 개월)간 변화 추이 - 양수=증가, 음수=감소를 나타내며 고객 행동 변화의 핵심 지표
추가 금액	- 시점별 거래 금액
추가 기본	- var21 - 주 거래 계좌 잔액 (고유값 많음)
추가 수량	- 거래 빈도
추가 잔액	- saldo_var2_ult1 - 최근 1 개월 특정 계좌 잔액
평균잔액(saldo_medio)	- 기간별 평균
최종 핵심	- var38 - 리스크 스코어 - (117310.979016494 특수값이 불만족 고객의 강력한 시그널, 최상위 중요 변수)

- TARGET - 고객 만족도 (0=만족 96%, 1=불만족 4%) - 예측 대상 변수

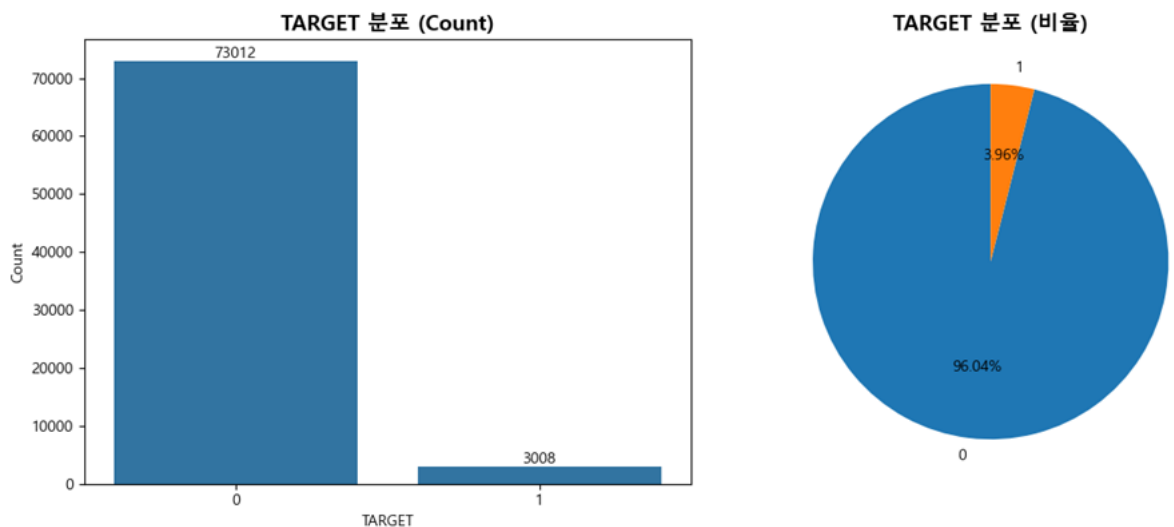
2.1.2.2 분석 및 시각화

1) 데이터 준비

- 데이터셋 : train.csv (약 7 만 6 천명 고객, 371 개 변수)
- 레이블 : TARGET
 - 0 : 만족 고객
 - 1 : 불만족 고객

2) 만족 vs 불만족 고객 비율

- 대부분 고객 만족(0) 상태 → 약 96% 이상
- 불만족 고객은 소수(약 4%) → 데이터 불균형 존재
- 시각화
 - 막대 그래프(bar chart)로 두 그래프 비율 비교

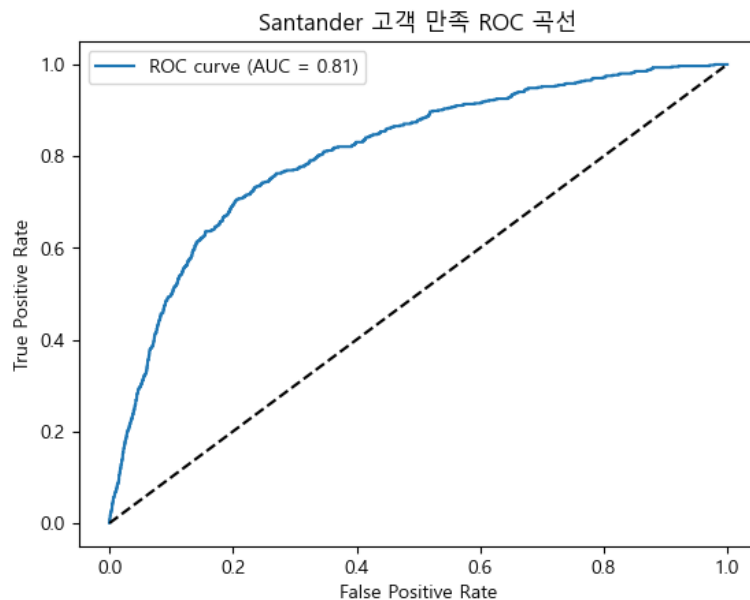


[그림 2.1.4 산탄데르 은행 캐글 고객 만족 분석 TARGET 비율]

3) ROC 곡선

- 모델 : XGBoost, LightGBM 등 트리 기반 알고리즘 사용
- 평가지표 : Accuracy 보다 ROC-AUC 가 적합 (불균형 데이터 때문)
- ROC 곡선 : 모델의 분류 성능을 시각적으로 확인 가능

- ROC 곡선 해석 :
 - X 축 (False Positive Rate, FPR)
 - 실제로는 만족 고객인데 불만족으로 잘못 분류한 비율
 - Y 축 (True Positive Rate, TPR)
 - 실제 불만족 고객을 올바르게 분류한 비율
 - ROC 곡선
 - 모델의 임계값을 변화시키면서 FPR 과 TPR 을 그린 곡선
 - 대각선(랜덤 분류기)
 - 성능이 거의 없는 모델은 (0,0)에서 (1,1)로 이어지는 대각선에 가까움
- AUC (AREA Under Curve) : ROC 곡선 아래 면적
 - 1.0 에 가까울수록 → 완벽한 분류
 - 0.5 에 가까울수록 → 랜덤 추측 수준



[그림 2.1.5 산탄데르 은행 고객 만족 ROC 곡선 시각화]

2.1.2.3 불균형 데이터셋에서의 ROC-AUC

- 클래스 비율에 대한 독립성
 - True Positive Rate (재현율 또는 민감도)와 False Positive Rate (1-특이도) 기반
 - 두 지표는 모두 실제 양성 샘플 또는 음성 샘플 내에서의 비율이므로, 전체 데이터셋 클래스 분포 변화에 영향을 덜 받는다.

- 모델의 변별력 종합 평가
 - AUC 값은 0 부터 1 까지의 범위에서 모델이 양성클래스와 음성클래스를 얼마나 잘 구별하는지 단일 수치로 요약
 - 다양한 임계값(threshold)에 걸친 성능을 보여주므로, 특정 임계값에 치우치지 않는 모델의 잠재적 성능 파악 가능
- 정확도(Accuracy)의 한계 극복
 - 데이터가 극심하게 불균형할 때는 단순히 다수 클래스를 예측하는 것만으로도 정확도가 높게 나올 수 있어, 정확도는 좋은 평가지표가 될 수 없다.
 - 반면, ROC-AUC 는 소수클래스에 대한 모델의 성능을 더 잘 반영한다.
- TPR/FPR 상충관계 시각화
 - 개발자가 특정 업무 목표에 맞춰 적절한 임계값을 선택할 수 있도록 도움

⇒ 불균형 데이터셋의 **모델 자체의 내재적인 변별 능력을 평가 및 비교**에 유용한 지표

2.1.2.4 데이터셋 Describe

- df.describe() 를 통해 각 변수의 평균, 표준편차, 최소/최댓값 등 기본 통계치 출력
- 단순히 평균·최소·최대값을 보는 것 이상으로, 데이터의 구조적 특성과 모델링 전략을 세우는 데 중요한 힌트를 파악 가능
- 데이터 타입
 - 정수형(int) 변수 : 약 260 개
 - 실수형(float)변수 : 약 111 개
- 익명 처리
 - 변수 이름에 의미가 없어, 직접적인 해석 불가
 - 따라서 ***EDA(탐색적 데이터 분석)** 와 ***Feature Selection** 이 매우 중요
- 클래스 불균형
 - 대부분 고객은 만족(0)
 - 하지만 불만족(1)은 소수 → 불균형 데이터 처리 필요

	ID	var3	var15	var38	TARGET
count	76020.000000	76020.000000	76020.000000	7.602000e+04	76020.000000
mean	75964.050723	-1523.199277	33.212865	1.172358e+05	0.039569
std	43781.947379	39033.462364	12.956486	1.826646e+05	0.194945
min	1.000000	-999999.000000	5.000000	5.163750e+03	0.000000
25%	38104.750000	2.000000	23.000000	6.787061e+04	0.000000
50%	76043.000000	2.000000	28.000000	1.064092e+05	0.000000
75%	113748.750000	2.000000	40.000000	1.187563e+05	0.000000
max	151838.000000	238.000000	105.000000	2.203474e+07	1.000000

8 rows × 371 columns

[그림 2.1.6 로드된 산탄데르 실습 데이터셋 데이터프레임 describe()]

1) 변수 분포와 스케일

- 일부 변수(var3, var15)는 **정수형**으로 범위가 작음 → 고객 나이, 카테고리형 변수일 가능성 존재
- 다른 변수(var38)는 **값의 범위가 매우 큼** → 고객 자산 규모와 유사, 로그 변환 필요
- 스케일 차이가 크면 모델 학습 시 특정 변수에 과도한 영향 → **정규화/표준화 필요**

2) 결측치 및 이상치 탐지

- describe() count 값이 전체 행 수 보다 작은 경우 → 결측치 존재
- min 과 max 값이 극단적으로 튀는 변수 → 이상치 가능성
 - e.g. 'var3' 에서 -99999 같은 값 발견 → 결측치 대체 코드로 사용된 것

3) 클래스 불균형 확인

- TARGET 변수의 mean ≈ 0.04 → 불만족 고객은 약 4%만 존재
- 이는 **심각한 클래스 불균형 문제** ⇒ 단순 Accuracy 는 무의미, **ROC-AUC** 또는 **F1-Score** 활용 필요

4) 변수 중요도 추정 힌트

- var15(나이로 추정)는 평균·표준편차가 현실적인 분포 → 고객 특성 반영 가능성이 높음
- var38 은 값이 크고 분산이 큼 → 자산 규모 관련 변수로 고객 만족과 연관될 가능성 있음
- describe() 만으로도 어떤 변수가 모델에 의미 있는 신호를 줄 수 있을지 가능 가능

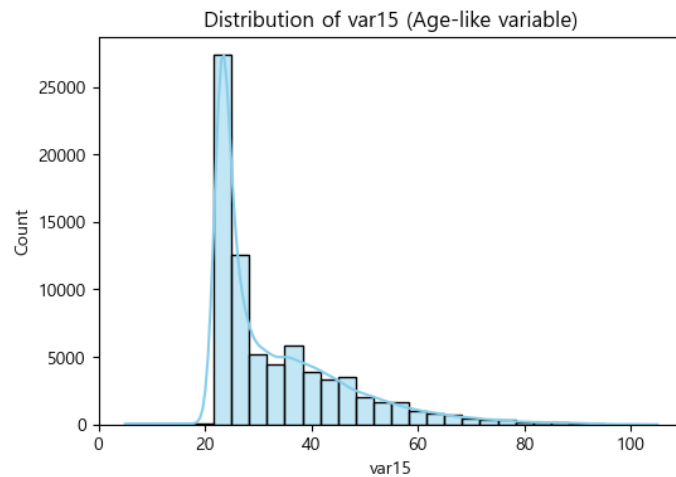
5) Feature Engineering 방향성

- 범위가 좁은 변수 : 원-핫 인코딩 또는 범주형 처리
- 범위가 넓은 변수 : 로그 변환, 스케일링
- 결측치 대체 : 평균/중앙값 대체, 혹은 특별한 값(-99999)을 결측치 인식 후 처리
 - 따라서 데이터셋 describe() 를 통해 결측치, 이상치, 스케일링 등 데이터 전처리 전략 수립 파악이 가능할 뿐 아니라, 불균형 데이터 처리 또는 변수 선택 등 모델링 접근 방향 또한 설정 가능하다.
 - 익명 변수라도 각 피쳐들의 분포를 통해 의미 또한 유추가 가능하다.

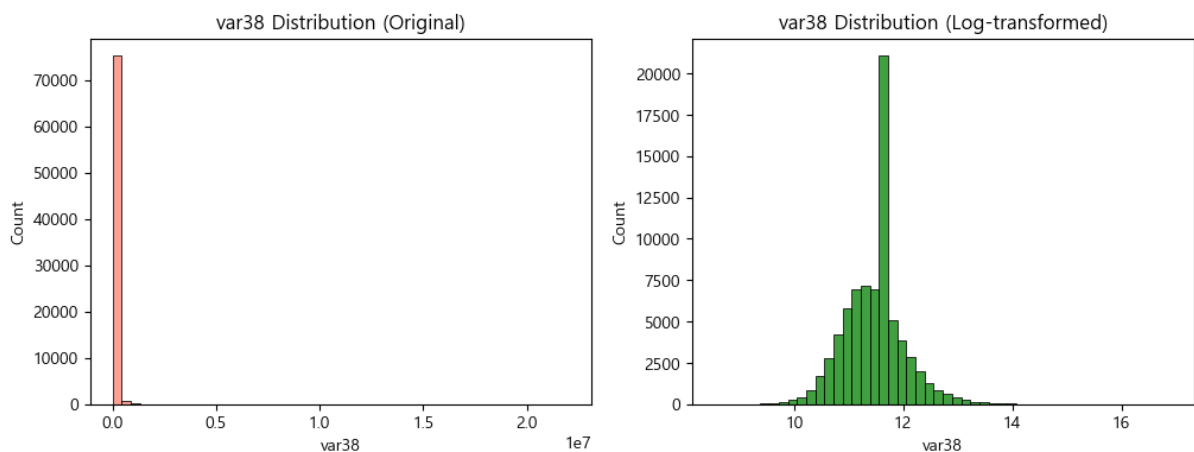
2.1.2.5 주요 피쳐분석

- 370 개의 익명화된 변수 중 고객 불만족을 가장 잘 설명하는 피쳐를 찾아내는 과정
- 이를 통해 모델 성능을 높이고, 은행이 고객 이탈을 예방할 수 있는 전략적 인사이트 도출
- 데이터 특성 이해
 - 데이터셋은 약 76,000 개의 피쳐로 구성되어있고, 모든 피쳐 이름은 익명화되어 있다.
 - 따라서, 단순히 변수 이름에 의존 불가능하며, 통계적 분포와 모델 기반 중요도 분석을 통해 의미를 추론해야한다.
- 불균형 클래스 문제
 - 위 데이터셋 분석 및 시각화, 그리고 describe 를 통해 대부분의 고객은 만족(0) 상태이고, 불만족(1) 고객은 소수임을 파악

- 따라서, 정확도(Accuracy)보다 ROC-AUC 같은 지표가 더 적합하며, 피쳐 분석은 소수 클래스 구분에 핵심적이다.
- 모델 해석 가능성 확보
 - 트리 계열 모델을 활용하여 피쳐 중요도를 계산
 - 이를 통해 산탄데르 은행은 어떤 요인이 고객 불만족을 유발하는지 설명할 수 있다.
 - describe() 결과를 기반으로 시각화를 하면 변수의 분포, 스케일 차이, 클래스 불균형을 직관적으로 확인 가능하다.
 - 특히, 피쳐 중 var15(나이 추정), var38(자산 규모 추정)을 시각화하여 데이터 전처리와 모델링 전략을 도출하는데 도움이 될 수 있었다.
 - 주요 피쳐들의 히스토그램 및 상관관계 Heatmap 으로 문제 파악
- var15(나이 추정) 히스토그램
 - describe() 에서 평균이 약 33, 최소 5, 최대 105 로 나타남
 - 20 ~ 40 대 구간에 데이터 집중되어 있고, 일부 극단값(100 세 이상)이 존재
 - 극단값은 이상치 처리나 그룹화 필요
- var38(자산 규모 추정) 히스토그램
 - describe() 에서 평균이 약 117,000, 최대 2,200,000 으로 매우 큰 값 존재
 - 치우친 긴 꼬리를 가진 형태(대부분 소규모, 일부 초고액 자산)
 - 로그 변환 후, 정규분포에 가까운 형태로 바뀌어 모델 학습에 안정성 확보
- TARGET(만족/불만족) 막대 그래프
 - describe() 에서 평균이 약 0.04 → 불만족 고객은 전체의 약 4%
 - 위 막대그래프에서 만족이 압도적으로 많고, 불만족은 소수인 것을 파악하였음
 - 클래스 불균형 문제 → SMOTE, 클래스 가중치, Precision-Recall Curve 활용

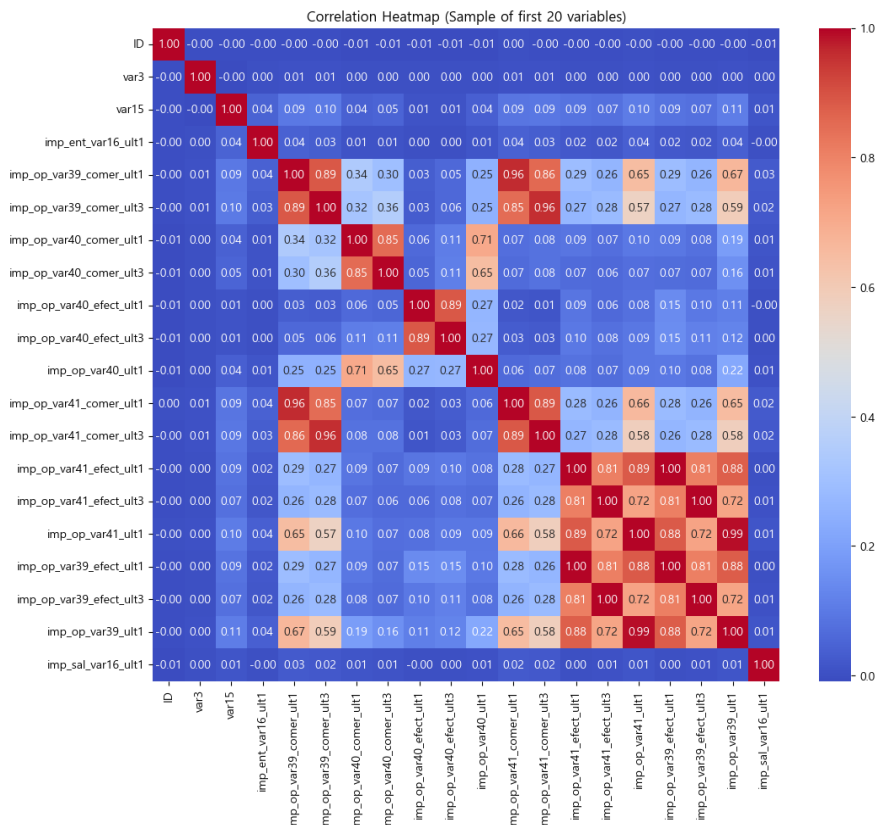


[그림 2.1.7 'var15(나이 추정)' 피쳐 히스토그램]



[그림 2.1.8 'var38(자산 규모 추정)' 히스토그램, 로그 변환 전후 비교]

- **var15** : 고객 나이로 추정되는 변수 → 분포 확인, 이상치 탐지
- **var38** : 자산 규모로 추정되는 변수 → 로그 변환 전후 비교로 분포 안정화 확인
- **TARGET** : 클래스 불균형 문제 → 적절한 평가 지표와 샘플링 기법 필요
- **상관관계 Heatmap** : 변수 간 상관관계 시각화 → 다중공선성 탐지, Feature selection 에 활용



[그림 2.1.9 상위 20 개 피쳐들의 상관관계 Heatmap 시각화]

2.1.2.6 주요 피쳐 분석을 통한 시사점

- 특정 변수들의 영향력
 - 익명화된 변수 중 일부는 불만족 고객을 강하게 구분하는 경향을 보였다.
 - e.g. 특정 피쳐 값이 극단적으로 치우친 경우 불만족 고객일 가능성이 높을 수 있다.
- 피쳐 상관관계
 - PCA 분석을 통해 변수 간 상관관계를 줄이고, 불만족 고객을 설명하는 주요 축을 도출 가능하다.
- 데이터 전처리의 중요성
 - 불만족 고객은 소수이므로 업샘플링이나 SMOTE 같은 기법을 활용하여 학습 데이터 균형을 맞추는 과정에서 어떤 피쳐가 잘 작동하는지 확인 필요하다.

- 산탄데르 은행의 실무적 인사이트
 - 고객 이탈 예방
 - 중요 피처를 중심으로 불만족 고객을 조기 탐지 → 맞춤형 대응 전략을 세울 수 있다.
 - 서비스 개선 포인트
 - 모델이 강조하는 변수들을 실제 서비스 개선지표로 활용 → 고객 만족도 상승
 - 설명 가능성 확보
 - 단순히 “모델이 불만족 고객이라고 예측했다”가 아니라, 어떤 요인 때문에 불만족 가능성이 높은지 설명이 되어야 한다.

2.2 탐색적 데이터 분석 (EDA)

2.2.1 통계적 요약

수치형 변수에 대한 기술통계 분석을 수행하여 데이터의 기본 분포와 특성을 파악하였다.

각 변수에 대해 평균, 중앙값, 표준편차, 최솟값, 최댓값, 사분위수를 계산하였으며, 이를 통해 변수 간

변동성, 이상치 존재 여부 및 데이터 전반적 스케일 확인하였다.

분석 결과 대부분의 변수는 비대칭 분포를 보였으며 일부 변수는 매우 큰 값 또는 0에 집중된 경향이 관찰되었다.

2.2.1.1 수치형 변수 기술통계

전체 데이터셋에 대한 수치형 변수의 기술통계를 수행하였으며, 특히 데이터의 분포 특성을 파악하기 위해 왜도를 분석하였다. 왜도는 데이터 분포의 비대칭성을 나타내는 지표로, 값이 0에 가까우면 대칭적 분포를, 양수 또는 음수일 경우 각각 오른쪽 또는 왼쪽으로 치우친 분포를 의미한다.

이 분석의 주요 목적은 데이터의 왜도 특성을 파악하여, 로그 변환(log transformation)이 적합한지 여부를 판단하는 데 있다. 로그 변환은 데이터의 분포를 정규분포에 가깝게 만들거나, 이상치의 영향을 줄이기 위해 자주 사용되는 기법이다. 특히, 왜도가 큰 경우 로그 변환이 효과적일 수 있으며, 이를 통해 분석의 정확성과 신뢰성을 높일 수 있다.

분석 결과, 전체 데이터 중 음수 값을 가지는 컬럼은 총 31개로 확인되었다. 이러한 컬럼들은 로그 변환이 어려운 특성을 가지고 있으며, 일반적으로 로그 변환은 양수 데이터에만 적용 가능하기 때문에, 별도의 처리 또는 변환 방법이 필요하다. 예를 들어, 음수 값을 양수로 변환하거나, 다른 비선형 변환 기법을 고려할 수 있다.

상위 20개 컬럼:

순위	컬럼명	왜도	최소값	최대값	Log변환
1	num_reemb_var33_ult1	275.72	0.00	3.00	가능
2	num_reemb_var17_hace3	275.72	0.00	3.00	가능
3	imp_trasp_var33_out_ult1	275.72	0.00	3000.00	가능
4	num_trasp_var33_out_ult1	275.72	0.00	3.00	가능
5	imp_reemb_var33_ult1	275.72	0.00	1200.00	가능
6	delta_num_trasp_var33_out_1y3	275.72	0.00	999999999.00	가능
7	delta_imp_trasp_var33_out_1y3	275.72	0.00	999999999.00	가능
8	saldo_medio_var29_hace3	275.72	0.00	145.20	가능
9	delta_num_aport_var33_1y3	275.72	-1.00	999999999.00	불가(음수)
10	delta_imp_reemb_var33_1y3	275.72	0.00	999999999.00	가능
11	delta_num_reemb_var33_1y3	275.72	0.00	999999999.00	가능
12	delta_imp_aport_var33_1y3	275.72	-1.00	999999999.00	불가(음수)
13	imp_reemb_var17_hace3	275.72	0.00	12027.15	가능
14	saldo_medio_var17_hace3	273.68	0.00	2368558.95	가능
15	imp_aport_var17_hace3	273.67	0.00	6083691.87	가능
16	saldo_var18	271.90	0.00	3000000.00	가능
17	saldo_var1	271.72	-0.90	3000000.00	불가(음수)
18	imp_amort_var18_ult1	270.33	0.00	15691.80	가능
19	saldo_medio_var17_hace2	269.33	-0.03	4210084.23	불가(음수)
20	saldo_var17	258.44	0.00	6119500.14	가능

[그림 2.2.1 왜도 분석]

이와 같은 분석은 데이터 전처리 과정에서 매우 중요한 단계로, 이후의 모델링 및 분석의 성과에 직결된다. 따라서, 왜도 분석과 함께 적절한 변환 방법을 선정하는 것은 데이터의 특성을 이해하고, 최적의 분석 결과를 도출하는 데 필수적이다.

```
=====
왜도(Skewness) 분석
=====
전체 컬럼 수: 369
왜도 > 3.0: 315개
왜도 > 5: 293개
왜도 > 10: 242개
```

[그림 2.2.2 왜도 컬럼 개수]

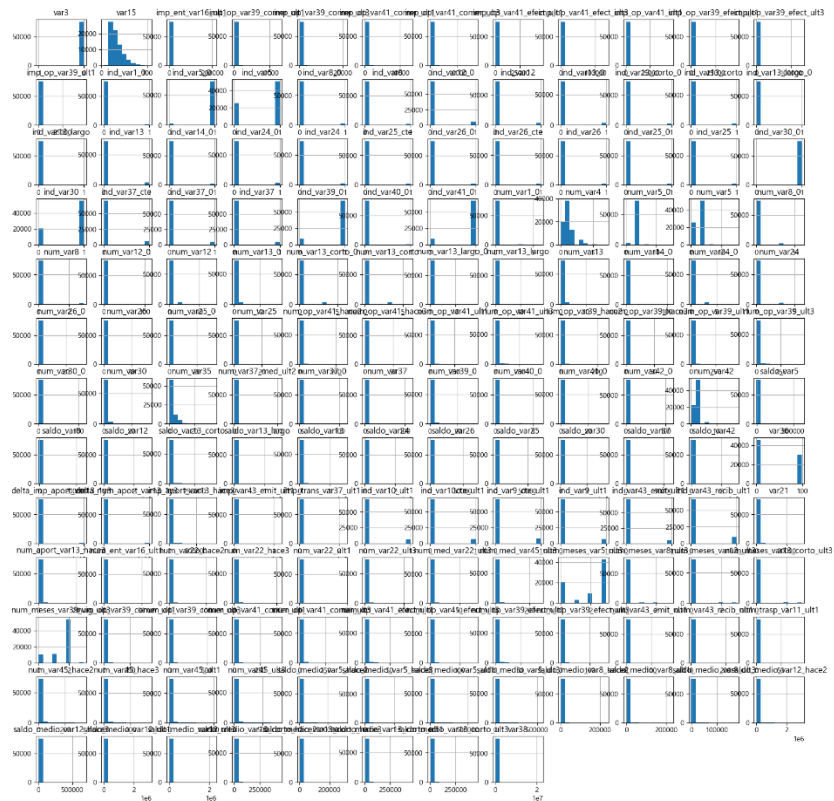
```
음수 값이 있는 컬럼: 31개
예시: ['var3', 'saldo_var1', 'saldo_var5', 'saldo_var8', 'saldo_var30']
```

[그림 2.2.3 음수 값 컬럼]

2.2.2 시각화 분석

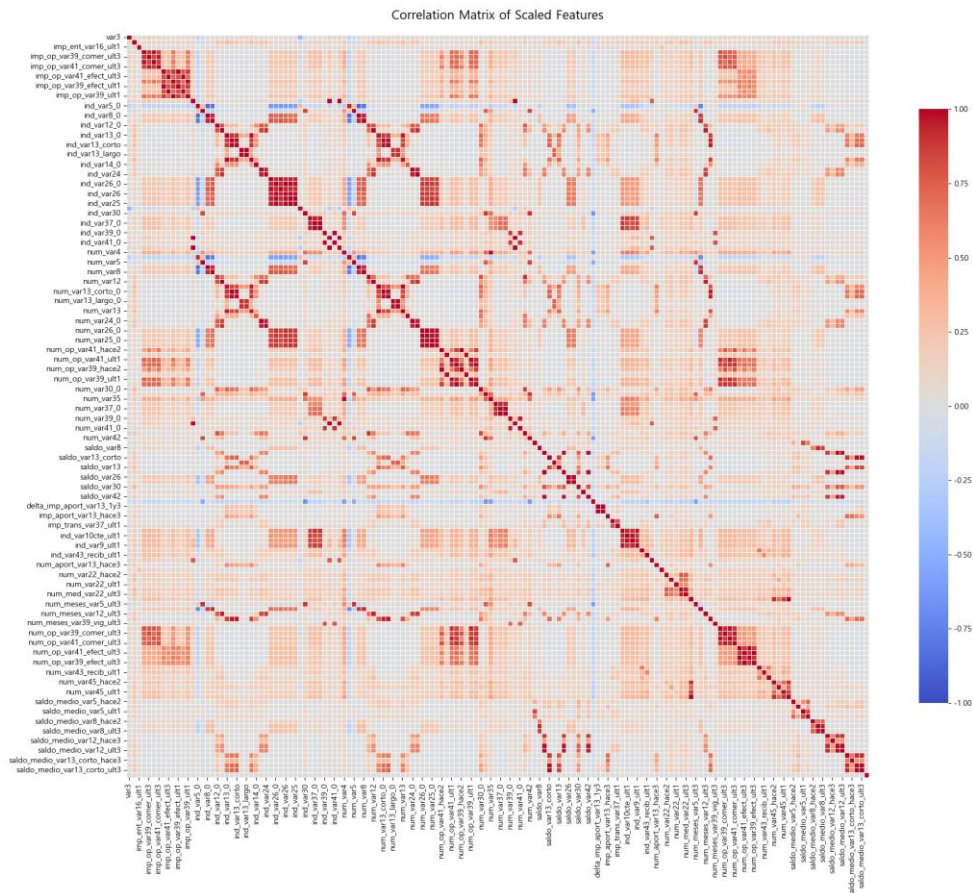
2.2.2.1 전체 컬럼 분포

전체 피처를 분석한 결과 대부분의 피처가 0에 편중된 희소 분포를 보이며, 일부 변수는 긴 꼬리를 가진 비정규 분포 형태가 확인되었다.



[그림 2.2.4 전체 피처 분포도]

2.2.2.2 피처 간 상관관계

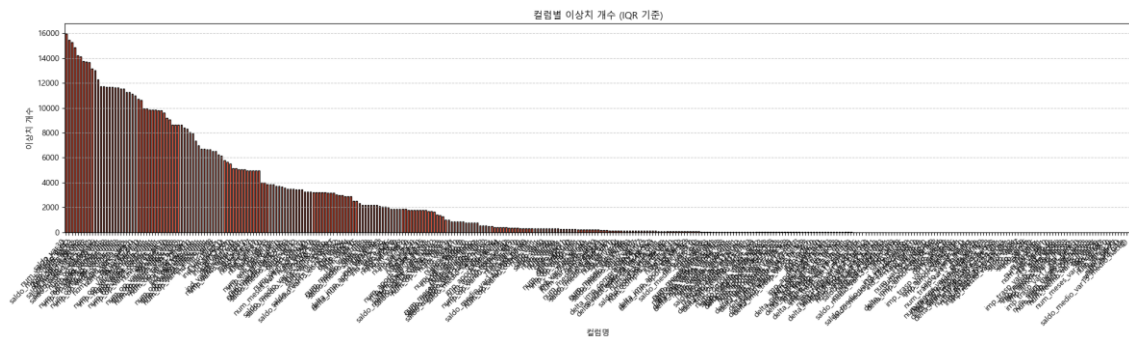


[그림 2.2.5 피처 상관관계]

상관행렬 히트맵을 통해 전체 변수 간 상관관계를 분석한 결과, 전반적으로 약한 상관성이 우세하지만 특정 변수 그룹에서는 강한 상관관계가 확인되었다. 특히 saldo, num, imp 계열 변수들은 유사한 패턴으로 묶여 나타나며 상관계수가 높은 군집을 형성하였다. 이는 해당 변수들이 동일한 계산 구조 또는 동일한 고객 행동 지표를 반영하고 있을 가능성이 있다.

2.2.2.3 이상치 탐지

각 피처의 이상치 분포를 확인하기 위해 시각화를 하였다.



[그림 2.2.6 이상치 분포도]

이 그래프는 각 변수(컬럼)에 이상치가 얼마나 존재하는지 비교한 차트이다.

여기서 이상치는 보통 값의 흐름에서 갑자기 튀거나 비정상적으로 보이는 값을 의미한다.

데이터의 일부 변수는 극단적인 값이 많아 추가적인 전처리가 필요하지만, 대부분의 변수는 안정적이어서 큰 수정 없이 모델에 활용할 수 있다.

2.2.3 발견된 문제점

2.2.3.1 결측치 패턴

	ID	var3
count	76020.000000	76020.000000
mean	75964.050723	-1523.199277
std	43781.947379	39033.462364
min	1.000000	-999999.000000
25%	38104.750000	2.000000
50%	76043.000000	2.000000
75%	113748.750000	2.000000
max	151838.000000	238.000000

[그림 2.2.7 var3 describe]

- var3 : 최소값 -999999 존재 → 암묵적 결측치

- var15 : 나이 컬럼에 100 세 이상 존재 → 결측 or 시스템 default 값일 가능성 ↑

2.2.3.2 이상치

```
count    76020.000000
mean      33.212865
std       12.956486
min        5.000000
25%       23.000000
50%       28.000000
75%       40.000000
max       105.000000
Name: var15, dtype: float64
var15
23      20170
24      6232
25      4217
26      3270
27      2861
...
97         3
104        2
98         1
105        1
101        1
Name: count, Length: 100, dtype: int64
100세 이상: 8
```

[그림 2.2.8 var15 100 세이상]

```
# var15 분포 확인
print(df['var15'].value_counts())

# 100 세 이상 값 개수
print("100 세 이상:", (df['var15'] > 100).sum())
```

var15 변수는 고객의 나이를 나타내는 것으로 추정되며, 전체 분포는 23~40 세 구간에 집중되어 있고 평균은 약 33.21, 중앙값은 28 세로 확인되었다. 또한 변수의 최댓값은 105 세로 나타났으며, 100 세 이상 값은 총 8 건으로 매우 드물게 나타나 이상치로 판단하였다.

```

var38
117310.979016    14868
451931.220000     16
463625.160000     12
104563.800000     11
288997.440000     11
236690.340000      8
329603.970000      7
125722.440000      7
67088.310000       7
128318.520000       7
Name: count, dtype: int64

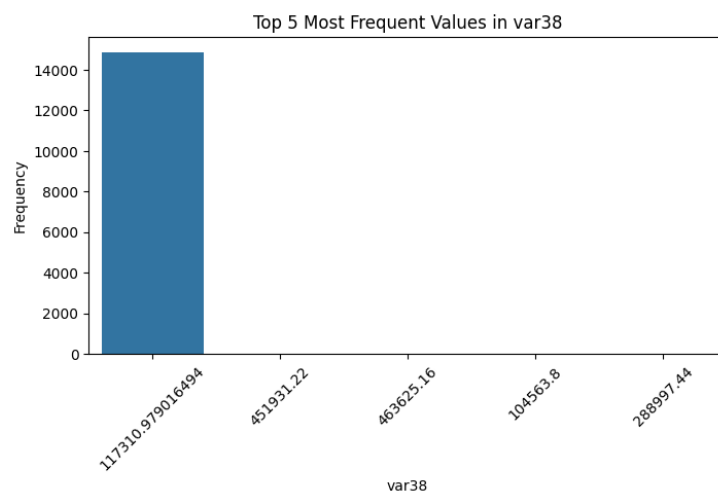
```

[그림 2.2.9 var38 중복개수]

```

# var38 패턴확인
X_features['var38'].value_counts().head(10)

```



[그림 2.2.10 var38 중복분포 차트]

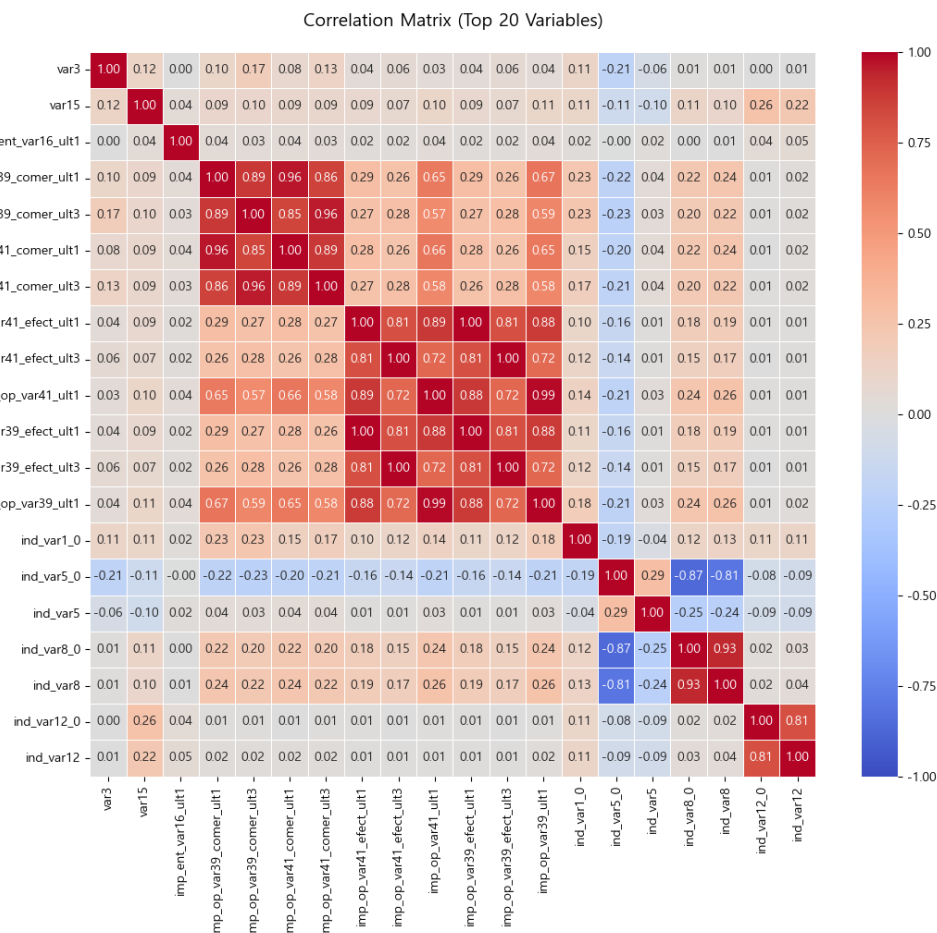
var38 변수의 값 분포를 분석한 결과, 특정 값(117,310.979016494)에 데이터가 집중되는 패턴이 확인되었다. 해당 값은 전체 데이터 중 약 14,868 건으로 가장 높은 빈도를 보이며, placeholder 값 또는 암묵적 결측치일 가능성을 시사한다

2.2.3.3 불균형 데이터

TARGET 값 분포를 확인한 결과, 클래스 0 이 약 96%, 클래스 1 이 약 4%로 나타나 매우 불균형한 형태를 보였다.

데이터의 클래스 비율이 약 96:4 로 심각한 불균형 특성을 보이므로, 단순 정확도(Accuracy)만으로 모델 성능을 평가할 경우 다수 클래스에 편향된 결과가 나타날 수 있다. 따라서 모델의 분류 임계값 변화에 따른 민감도를 반영하고, 두 클래스 간 판별 능력을 종합적으로 평가할 수 있는 ROC-AUC 지표를 주요 성능 평가 지표로 사용하였다. ROC-AUC 는 불균형 환경에서도 모델의 진짜 분류 능력을 비교적 안정적으로 측정할 수 있다는 점에서 적합한 평가 기준이 된다.

2.2.3.4 다중공선성



[그림 2.2.11 TOP20 상관관계 히트맵]

상관계수 분석 결과 imp, num, saldo 계열 변수들이 서로 강하게 연결된 군집 형태로 나타났으며, 이는 서로 유사한 정보를 공유하고 있는 변수들이 존재한다는 의미로, 데이터셋 내 다중공선성이 존재함을 보여준다.

다중공선성은 모든 모델에 동일한 영향을 미치는 것은 아니다. 트리 기반 모델(Random Forest, XGBoost, LightGBM 등)은 분할 기준에 따라 변수 중요도를 선택적으로 반영하기 때문에 다중공선성의 영향이 상대적으로 적다. 반면, 선형 계열 모델(Logistic Regression, Linear SVC, SVM 등)에서는 계수 불안정성, 해석력 저하, 과적합 가능성이 증가할 수 있어 더 민감하게 영향을 받는다.

2.2.3.5 기타

피처 분포 특성을 조사한 결과, 대부분의 변수에서 0 값이 높은 비율을 차지하며, 이는 데이터셋이 높은 희소성(Sparsity)을 갖고 있음을 보여준다. 이러한 0 값은 단순 비사용 항목이 아닌 고객의 서비스 참여도 및 금융 활동 수준을 의미하는 행동적 의미를 갖는다. 또한 var15 와 saldo 계열과 같은 연속형 변수에서는 분포가 한쪽으로 치우친 비정규적 형태가 관찰되어 변환 또는 이상치 처리가 필요한 것으로 판단된다.

2.3 데이터 전처리

본 절에서는 산탄데르 은행 고객 만족도 예측 모델 구축을 위해 수행한 데이터 전처리 과정을 연구 보고서 수준의 서술형 구조에 따라 상세히 기술한다. 본 데이터는 371 개의 고차원 피처로 구성되어 있으며, 구조적 결측치, 0 값 편중, 스케일 차이, 다중공선성 등 다양한 품질 이슈를 포함하고 있었다. 따라서 모델 성능과 안정성을 확보하기 위해서는 체계적이고 단계적인 전처리 전략이 필수적이었다. 이에 따라 본 연구에서는 데이터 정제, 결측치 및 이상치 처리, 의미 없는 피처 제거, 고왜도 보정, 다중공선성 해결, 인코딩 및 스케일링 등을 포함한 다음의 절차를 수행하였다.

[표] 전처리 전체 단계 요약

단계	주요 내용
1. 데이터 정제	비표준 값 처리 및 결측치 치환 전략 적용 ⁷
2. 피처 선택 (1)	Zero-value 비율 분석에 따른 피처 제거 ⁸
3. 이상치 처리	이상치 탐지 및 값 보정 ⁹
4. 변수 변환	고 왜도 변수에 대한 로그 변환 적용 ¹⁰
5. 피처 선택 (2)	다중공선성 해소를 위한 상관관계수 기반 피처 제거 ¹¹
6. 인코딩/스케일링	필요 시 인코딩 적용 및 스케일 정규화 ¹²
7. 최종 확정	최종 Feature 목록 확정 ¹³

2.3.1 결측치 처리

```
def split_features_target(train_df, target_col="TARGET"):
    X = train_df.drop(["ID", target_col], axis=1)
    y = train_df[target_col]
    return X, y
```

데이터셋을 탐색한 결과, 일부 변수에서 명시적으로 결측을 나타내는 비정상 값이 확인되었다. 특히 var3 변수에서는 -999999 와 같은 값이 반복적으로 관측되었으며, 이는 실제 고객 데이터라기보다는 데이터 수집 과정에서 결측치를 임의의 숫자로 대체한 것으로 판단되었다. 이러한 값은 모델 학습 시 특정 패턴으로 오인될 가능성이 있으므로 최빈값으로 치환하여 데이터의 일관성을 확보하였다. 또한 V13 변수 내 -9999 값이 116 건 존재하였으며, 이는 결측치로 판단되어 동일한 방식으로 처리하였다.

2.3.2 Zero 값의 의미 파악 및 처리

```
# var3 처리

X_features['var3'] = X_features['var3'].replace(-999999, 2)
X_test['var3'] = X_test['var3'].replace(-999999, 2)
```

데이터셋 전반에 걸쳐 0 값이 지나치게 높은 비율로 나타나는 변수가 존재하였다. 이들 값이 실제 의미를 가지는지 혹은 결측치를 대체한 값인지를 판단하기 위해 각 피처의 zero_count_rate 를 분석하였다. 검토 결과, 전체 값 중 99% 이상이 0 으로 구성된 변수들은 정보량이 거의 없으며, 모델의 분류 성능 향상에 기여하지 못한다고 판단하였다. 따라서 이러한 피처 220 개를 제거하였으며, 제거 목록은 별도의 파일로 정리하였다.

2.3.2 상관관계 기반 피처 제거

```
# 상관관계 처리

def remove_zero_columns2(train_df, test_df, threshold_rate=0.99, save_report=True):
    summary_data = []

    for col in train_df.columns:
        zero_count = (train_df[col] == 0).sum()
        zero_rate = zero_count / len(train_df)
        mode_freq = train_df[col].value_counts().iloc[0]
        mode_rate = mode_freq / len(train_df)
```

```

summary_data.append({
    "ColumnName": col,
    "na_Sum": train_df[col].isna().sum(),
    "nUnique": train_df[col].nunique(),
    "mode": train_df[col].mode()[0],
    "modeFreq": mode_freq,
    "modeFreqRate": f"{mode_rate * 100:.2f}%",
    "zero_count": zero_count,
    "zero_count_rate": zero_rate
})

summary_df = pd.DataFrame(summary_data)

remove_cols = summary_df[summary_df["zero_count_rate"] >
threshold_rate]["ColumnName"].tolist()

clean_train_df = train_df.drop(remove_cols, axis=1)
clean_test_df = test_df.drop(remove_cols, axis=1)

return clean_train_df, clean_test_df

```

다중공선성 문제를 해결하기 위해 피처 간 상관관계를 계산한 후, 상관계수 절댓값이 0.95 이상인 변수들을 제거하였다. 이러한 변수들은 동일한 정보를 중복하여 제공하고 있었으며, 모델 학습 시 불필요한 복잡도 증가와 과적합을 초래할 수 있었다. 그 결과 총 53 개의 변수가 제거되었고, 최종적으로 96 개의 피처로 축소되었다.

2.3.3 이상치 처리

```

X_features['var15'] = X_features['var15'].clip(upper=100)

```

연속형 변수 중 일부는 비정상적으로 큰 값이나 데이터 분포를 편향시키는 극단값을 포함하고 있었다. 예를 들어 var15 변수는 고객의 나이를 의미하는 것으로 보였으나, 100 세를 초과하는 값들이 발견되었다. 이러한 값은 관측 오류 가능성이 높으며 분포의 꼬리를 과도하게 늘어뜨릴 위험이 있어, 상한값을 100 으로 제한하였다.

2.3.4 로그 변환을 통한 고왜도 보정

금융 데이터 특성상 특정 변수들이 극단적으로 치우친 분포를 나타내는 경우가 많았다. 왜도(skewness) 절댓값이 높은 변수는 모델의 결정 경계 형성을 왜곡할 수 있기 때문에 이러한 피처들에 대해 log1p 변환을 적용하였다. 분석 결과 총 71 개 변수가 변환 대상으로 선정되었으며, 변환 후 분포가 안정적으로 개선되었다.

2.3.5 인코딩

데이터셋은 대부분 수치형 변수로 구성되어 있어 대규모 인코딩 작업은 필요하지 않았으나, 범주형 변수가 존재할 경우 모델 특성에 따라 One-Hot Encoding 또는 Label Encoding 을 적용할 수 있다. 본 프로젝트에서는 범주형 변수의 영향이 미미하다고 판단하여 추가 인코딩을 수행하지 않았다.

2.3.6 스케일링

트리 기반 알고리즘은 변수의 스케일에 민감하지 않지만, 로지스틱 회귀나 SVM 과 같은 모델은 입력값의 크기 차이에 영향을 받는다. 따라서 모델 간 공정한 비교를 위하여 StandardScaler 를 적용하여 모든 변수의 스케일을 정규화하였다.

2.3.8 최종 Feature 요약

전처리 전 과정은 원본 데이터(371 개 피처)에서 불필요하거나 품질 저하를 유발하는 요소를 제거하고, 변수의 통계적 특성과 모델 적합성을 고려하여 의미 있는 피처만을 남기기 위한 절차였다. 그 결과 최종적으로 96 개 피처가 선정되었으며, 이는 모델 학습의 효율성 향상뿐 아니라 과적합 방지와 해석 가능성 제고에도 기여하였다.

이처럼 단계적으로 전처리를 수행한 결과, 초기 데이터의 약 74%에 해당하는 피처가 정제되거나 제거되었으며 최종적으로 96 개의 핵심 변수만이 모델링에 활용되었다.

2.4 모델링

2.4.1 학습/검증 데이터 분할

2.4.1.1 Train/Validation/Test 비율

- Train: Kaggle 에서 제공한 `train.csv` 중 80%를 **학습용(Train)** 으로 사용하였다.
(60,816 건)
- Validation: 나머지 20%를 **검증용(Validation)** 으로 사용하여 모델 선택과 하이퍼파라미터(HyperParameter) 튜닝에 활용하였다. (15,204 건)
- Test: 레이블이 없는 `test.csv` 는 **최종 제출용 데이터**로만 사용하였으며, 내부 성능 비교에는 포함하지 않았다.

2.4.1.2 분할 방법

- **층화 분할(Stratified Split) 적용**
타깃 변수 `TARGET` 의 1 클래스 비율이 매우 낮은 **심각한 불균형 데이터**이기 때문에, `train_test_split(..., stratify=TARGET)` 을 사용하여 **Train/Validation** 에 **동일한 클래스 비율이 유지**되도록 분할하였다.
- **여러 비율 실험 후 8:2 로 확정**
초기에는 5:5 분할(추가 Temp 세트 활용)과 7:3 분할 등 여러 안을 실험하였으나, ROC-AUC 와 F1-Score 가 가장 안정적으로 나오고, 학습 데이터도 충분히 확보되는 8:2 분할을 최종적으로 채택하였다.
- **언더샘플링 적용 위치**
클래스 불균형 보정을 위해 **언더샘플링은 Train 데이터에만 적용**하고, Validation/Test 데이터는 원래 분포를 그대로 유지하여 실제 환경과 유사한 평가가 가능하도록 하였다.

2.4.1.3 Cross-Validation 전략

```
stacking_model = StackingClassifier(
    estimators      = [('rf', rf_clf), ('xgb', xgb_clf), ('lgbm', lgbm_clf)],
    final_estimator = meta_model,
    cv              = StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    n_jobs          = -1
)
```

- **StratifiedKFold 기반 교차검증 :**

하이퍼파라미터 튜닝과 모델 비교를 위해 `**StratifiedKFold(n_splits=5, shuffle=True, random_state=42)**`를 사용하였다.

- **불균형 데이터 대응 :**

각 Fold마다 `TARGET`의 0/1 비율이 전체 데이터와 유사하게 유지되도록 층화(Stratified) 설정을 적용하여, 소수 클래스(1)도 매 Fold마다 일정 비율로 포함되도록 하였다.

- **튜닝과의 연결 :**

HyperOpt 및 GridSearchCV에서 각 파라미터 조합에 대해 **5-Fold ROC-AUC** 평균을 **Objective**로 사용하여, 단일 Train/Validation 분할에 의존하지 않는 안정적인 성능 평가를 수행하였다.

2.4.2 베이스라인 모델

2.4.2.1 사용 모델

- 가장 단순한 기준선을 설정하기 위해 `scikit-learn`의 `DummyClassifier`를 사용하였다.

`strategy="most_frequent"`를 사용하여, 학습 데이터에서 가장 많이 등장하는 클래스로만 예측하도록 설정하였다.

이를 통해 "아무 것도 학습하지 않은 모델"의 성능을 기준선으로 두고, 이후 모델들이 얼마나 개선되는지 비교하였다.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
dummy_clf = DummyClassifier(strategy="most_frequent", random_state=42)
dummy_clf.fit(X_train, y_train)
y_pred = dummy_clf.predict(X_test)
print("DummyClassifier ROC-AUC:", roc_auc_score(y_test, y_pred))
print("DummyClassifier Accuracy:", accuracy_score(y_test, y_pred))
print("DummyClassifier Precision:", precision_score(y_test, y_pred))
print("DummyClassifier Recall:", recall_score(y_test, y_pred))
print("DummyClassifier F1 score:", f1_score(y_test, y_pred))
print("DummyClassifier Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

2.4.2.2 성능 지표

```
DummyClassifier ROC-AUC: 0.5
DummyClassifier Accuracy: 0.9604051565377533
DummyClassifier Precision: 0.0
DummyClassifier Recall: 0.0
DummyClassifier F1 score: 0.0
DummyClassifier Confusion Matrix:
[[14602    0]
 [  602    0]]
```

[그림 2.4.2.2 DummyClassifier 성능 지표]

Accuracy: 0.96

- ✓ DummyClassifier 는 대부분의 샘플을 0(만족)으로 예측하기 때문에, Accuracy 는 약 0.96 수준으로 높게 나타났다.

ROC-AUC: 0.50

- ✓ ROC-AUC 는 약 0.5 수준(랜덤 예측과 유사) 으로, 실질적인 분류 능력은 거의 없다는 것을 보여준다.

F1 score / Precision / Recall : 0.0

- ✓ 소수 클래스인 1 에 대해 Recall 과 F1-score 는 거의 0 에 가깝게 측정되어, 불만족 고객을 사실상 찾아내지 못하는 모델임을 확인하였다.

Confusion Matrix : 모든 샘플을 0 으로 예측하여, 1 을 거의 못 잡는 것을 확인하였다.

2.4.2.3 결과

Baseline 에 의하면, Accuracy 는 좋아 보이지만, 실제로는 class 1(불만족) 을 거의 못 잡는 것을 알 수 있다.

Accuracy 의 한계 확인

DummyClassifier 가 아무 것도 학습하지 않은 상태에서도 Accuracy 0.9 이상을 기록하는 것을 통해, 불균형 데이터에서 Accuracy 지표만으로는 모델의 품질을 판단하기 어렵다는 점을 확인하였다.

주요 평가지표 재선정

따라서 본 프로젝트에서는 ROC-AUC 를 주요 평가 지표로 삼고, 소수 클래스(불만족 고객)를 잘 찾아내는지를 보기 위해 Recall 과 F1-score 를 보조 지표로 함께 사용하기로 하였다.

2.4.3 모델 선정

2.4.3.1 Base Model – XGBoost

Kaggle 상위권 솔루션들이 공통적으로 사용하는 모델이라는 점과, 불균형 데이터에서의 높은 성능·정규화·조기 종료(early stopping) 기능을 고려하여 XGBoost 를 베이스라인 모델로 선정하였다.

2.4.3.2 트리 계열 – LightGBM, RandomForest

- LightGBM 은 XGBoost 와 유사한 성능을 보이면서도 학습 속도가 빠르고 대용량 데이터에 적합하다는 장점이 있어 비교 대상으로 포함하였다.
- RandomForest 는 상대적으로 단순하지만 안정적인 앙상블 트리 모델로, 다른 트리 계열 모델과의 성능 · feature importance 패턴 비교를 위해 사용하였다.

2.4.3.3 선형 모델 – Logistic Regression

해석 가능성이 높고 구현이 간단하다는 장점 때문에 Logistic Regression 을 추가로 실험하여, 트리 계열 모델과의 성능 차이 및 중요한 변수의 차이를 비교하였다.

2.4.3.4 앙상블 확장 – Stacking

회의에서 논의한 대로, 개별 모델의 장점을 결합하기 위해 이후 XGBoost, LightGBM, RandomForest, Logistic Regression 등을 1 단계 모델로 사용하고, Logistic Regression 을 메타 모델로 하는 스택킹 앙상블을 구성하였다.

2.4.4 하이퍼파라미터 튜닝

2.4.4.1 튜닝 방법

HyperOpt : 베이지안 최적화, 효율적 탐색

- XGBoost 와 LightGBM 등 주요 트리 기반 모델의 하이퍼파라미터는 HyperOpt 의 TPE 알고리즘을 이용해 자동 탐색하였다.

교차검증 기반 : Objective

- 각 파라미터 조합에 대해 Stratified 5-fold 교차검증을 수행하고, 평균 ROC-AUC 가 최대가 되도록 파라미터를 최적화하였다.

단순 모델의 경우 : Grid/Random Search

- Logistic Regression 과 RandomForest 등 비교적 단순한 모델은 GridSearchCV 또는 RandomizedSearchCV 를 사용해 소수의 핵심 파라미터(C, max_depth, n_estimators 등)를 탐색하였다.

2.4.4.2 탐색 범위

XGBoost

```
search_space = {'max_depth': hp.quniform('max_depth', 3, 10, 1),
                'learning_rate': hp.uniform('learning_rate', 0.01, 0.3),
                'n_estimators': hp.quniform('n_estimators', 100, 500, 10),
                'subsample': hp.uniform('subsample', 0.6, 1.0),
                'colsample_bytree': hp.uniform('colsample_bytree', 0.6, 1.0),
                'reg_lambda': hp.uniform('reg_lambda', 0, 5),
                'reg_alpha': hp.uniform('reg_alpha', 0, 5)
                }
```

[표 4.3.1 XGBoost 하이퍼 파라미터 범위]

XGBoost	
'max_depth'	3~10
'learning_rate'	0.01~0.3
'n_estimators'	100~500
'subsample'	0.6~1.0
'colsample_bytree'	0.6~1.0
'reg_lambda'	0~5
'reg_alpha'	0~5

LightGBM

```
search_space_lgbm = {
```

```

'n_estimators':    hp.quniform('n_estimators', 200, 800, 50),
'learning_rate':   hp.loguniform('learning_rate', -4, -1.5),    # 0.018~0.223
'max_depth':       hp.quniform('max_depth', 3, 10, 1),
'num_leaves':      hp.quniform('num_leaves', 16, 60, 4),
'min_child_samples': hp.quniform('min_child_samples', 5, 30, 1),
'subsample':       hp.uniform('subsample', 0.6, 1.0),
'colsample_bytree': hp.uniform('colsample_bytree', 0.6, 1.0),
'class_weight':    hp.choice('class_weight', [None, {0:1, 1:2}, {0:1, 1:3}, {0:1, 1:4}])
}

```

[표 4.3.2 LightGBM 하이퍼 파라미터 범위]

LightGBM	
'max_depth'	3~10
'learning_rate'	0.01~0.22
'n_estimators'	200~800
'subsample'	0.6~1.0
'colsample_bytree'	0.6~1.0
'num_leaves'	16~60
'min_child_samples'	5~30
'class_weight'	1:2 / 1:3 / 1:4

RandomForest

```

search_space_rf = {
    'n_estimators':    hp.quniform('n_estimators', 100, 500, 50),
    'max_depth':       hp.quniform('max_depth', 6, 30, 1),
    'min_samples_split': hp.quniform('min_samples_split', 2, 15, 1),
    'min_samples_leaf':  hp.quniform('min_samples_leaf', 1, 8, 1),
}

```

```

'max_features': hp.choice('max_features', ['sqrt', None]),
'class_weight_idx': hp.choice('class_weight_idx', [0, 1, 2, 3])
}

```

[표 4.3.3 RandomForest 하이퍼 파라미터 범위]

RandomForest	
'max_depth'	6~30
'n_estimators'	100~500
'max_features'	'sqrt'
'min_samples_leaf'	1~8
'min_samples_split'	2~15
'class_weight'	1:2 / 1:3 / 1:4

2.4.4.3 최적 파라미터

XGBoost

```

xgb_best_params = {
    "random_state": 23,
    "n_estimators": 320,
    "colsample_bytree": 0.88,
    "gamma": 0.058,
    "learning_rate": 0.13,
    "max_depth": 6,
    "scale_pos_weight": 10,
    "min_child_weight": 2,
    "subsample": 0.85,
    "eval_metric": "auc",
    "use_label_encoder": False,
    "n_jobs": -1,
}

```

```
}
```

LightGBM

```
lgbm_best_param = {
    'random_state' : 23,
    'n_estimators' : 400,
    'num_leaves' : 36,
    'learning_rate' : 0.03,
    'subsample' : 0.9,
    'colsample_bytree' : 0.75,
    'reg_alpha' : 0.6,
    'reg_lambda' : 0.2,
    'class_weight' : {0:1, 1:10},
    'n_jobs' : -1
}
```

RandomForest

```
rf_best_param = {
    "random_state": 23,
    "n_estimators": 390,
    "max_depth": 25,
    "class_weight": {0: 1, 1: 2},
    "min_samples_leaf": 1,
    "min_samples_split": 7,
    "n_jobs": -1
}
```

LogisticRegression

```
meta_best_params = {
```

```

    "random_state": 23,
    'C': 0.029,
    'max_iter': 1000,
    'penalty': 'l2',
    'solver': 'lbfgs',
    'class_weight': 'balanced',
    "n_jobs": -1
}

```

2.4.5 모델 성능 비교

모델	ROC-AUC (Validation)	학습 시간
DummyClassifier	0.5	빠름
XGBoost (default)	0.8509	중간
XGBoost (tuned)	0.8402	느림
LightGBM (default)	0.8438	빠름
LightGBM (tuned)	0.8452	중간
LogisticRegression	0.8051	빠름
RandomForest	0.8422	느림

분석:

- 하이퍼파라미터 튜닝으로 소폭 향상
- XGBoost 와 LightGBM 성능 유사

· 디폴트 값도 나쁘지 않음

제공된 test.csv에는 정답 레이블이 포함되어 있지 않기 때문에, 본 프로젝트에서는 내부적으로 분할한 Validation 데이터와 교차검증 결과를 기준으로 모델을 비교하였다. 따라서 성능 비교 표에는 ROC-AUC(Test) 대신 ROC-AUC(Validation)만을 제시한다.

2.5 모델 평가

2.5.1 평가지표

2.5.1.1 주요 평가 지표 선정 사유

- ROC-AUC: 불균형 데이터에 적합, 임계 값 독립적
- 재현율(Recall): 실제 불만족 고객을 얼마나 잘 찾아 내는가
- 정밀도(Precision): 불만족으로 예측한 것 중 실제 불만족 비율
- F1 Score: Precision 과 Recall 의 조화 평균

불균형데이터구조의 데이터를 DummyClassifier 에 넣어 Accuracy(정확도)를 측정하게 되면, 학습을 안 한 상태에서도 정확도가 0.96 까지 측정된다.

```
train_df = pd.read_csv('../data/train.csv')
X = train_df.drop('TARGET', axis=1)
y = train_df['TARGET']

# 3. 학습/검증 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. DummyClassifier 생성 및 학습
dummy_clf = DummyClassifier(strategy="most_frequent", random_state=42)
dummy_clf.fit(X_train, y_train)

# 5. 예측 및 평가
y_pred = dummy_clf.predict(X_test)
print("DummyClassifier Accuracy:", accuracy_score(y_test, y_pred))
```

결과 :

```
DummyClassifier Accuracy: 0.9604051565377533
```

[그림 2.5.1.1 더미분류 정확도]

2.5.1.2 각 지표별 결과

모델명	데이터 전처리	ROC-AUC	F1-Score	Recall
XGBoost	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용	0.8509	0.2974	0.515
LightGBM	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용	0.8438	0.0099	0.005
LogisticRegression	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용	0.8051	0.0129	0.0066
RandomForest	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용	0.8422	0.0163	0.0083

모든 모델에서 ROC-AUC 는 높지만 F1-Score 와 Recall 이 극도로 낮다. 이는 **심각한 클래스 불균형**을 나타낸다

현재 추가적인 데이터 전처리가 필요한 것으로 확인된다

추가적인 전처리로 상관관계 비율이 0.95 이상인 피쳐 삭제, 행 전체가 0 인피쳐 삭제를 진행했다

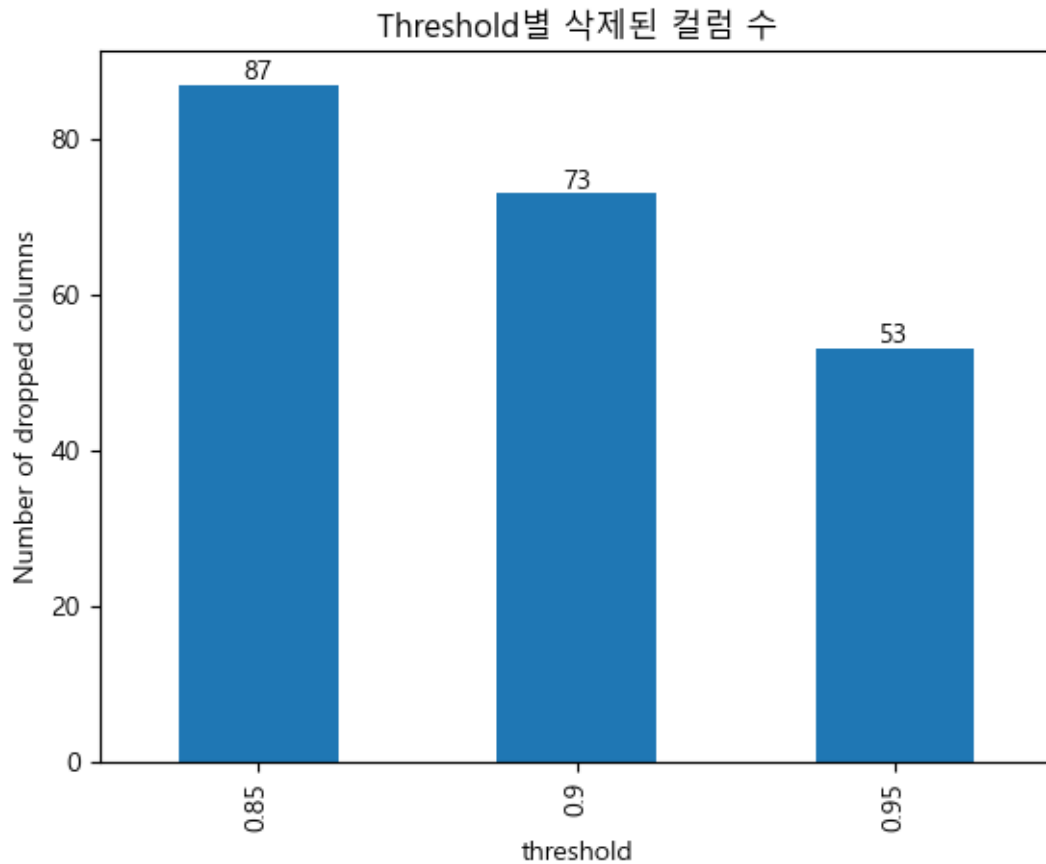
모델명	데이터 전처리	ROC-AUC	F1-Score	Recall
XGBoost	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용 행전체가 0 인 피쳐 삭제, corr 비율 0.95 이상인 피쳐 삭제	0.852	0.0066	0.0033
LightGBM	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용 행전체가 0 인 피쳐 삭제, corr 비율 0.95 이상인 피쳐 삭제	0.8452	0.2870	0.5598
LogisticRegression	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용 행전체가 0 인 피쳐 삭제, corr 비율 0.95 이상인 피쳐 삭제	0.8063	0.1617	0.7724
RandomForest	Var3 이상치를 최빈값 2 로 대체 StandardScaler 적용 행전체가 0 인 피쳐 삭제, corr 비율 0.95 이상인 피쳐 삭제	0.8263	0.2653	0.3953

- 추적인 데이터 전 처리 진행 후 대부분의 모델은 ROC-AUC 는 감소하였으나 F1-Score 와 recall 에서는 유의미한 성능 향상이 되었다.
- 거의 모든 모델에서 양성 탐지 능력이 향상되었다.
- 피쳐 수를 감소시킴으로 과적합을 방지하였다.

XGBoost 의 비정상적 수치 하락

가능한 원인:

1. 하이퍼파라미터 미조정: 전처리 후 피쳐 수 감소에 맞춘 재튜닝을 안 하였다
2. Threshold 문제: 예측 임계값이 너무 높게 설정되었을 수 있다
3. Class Weight 미적용: 불균형 처리 없이 학습했을 수 있다
4. 데이터 누수: 전처리 과정에서 중요 정보 손실이 있을 수 있다



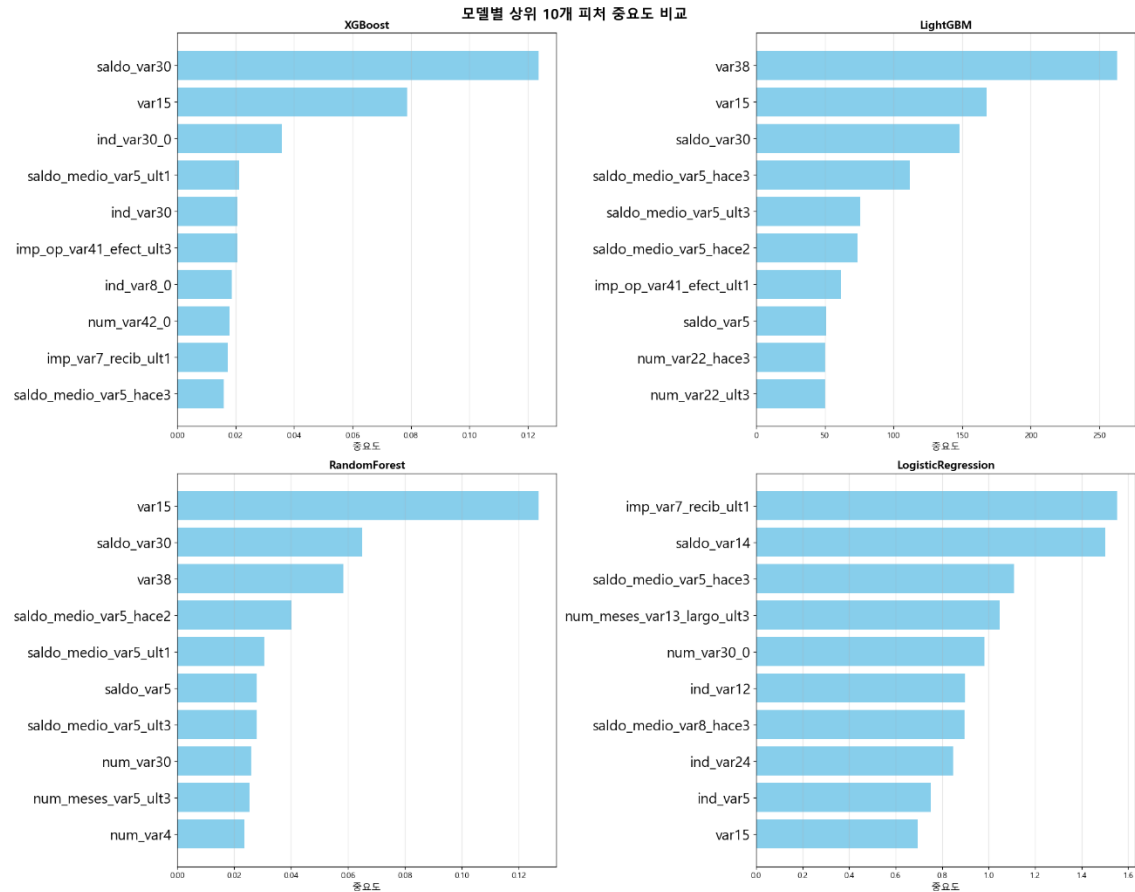
[그림 2.5.1.2 threshold 별 삭제 컬럼 수]

- 정보성이 높은 피처만 남겼으므로 일반화 능력이 향상되었다.

모델	ROC-AUC 변화	F1-Score 변화	Recall 변화
XGBoost	0.8509 → 0.8510 (↑ 0.0001)	0.2974 → 0.0066 (↓ 0.2908)	0.5150 → 0.0033 (↓ 0.5117)
LightGBM	0.8438 → 0.8452 (↑ 0.0014)	0.0099 → 0.287 (↑ 0.2771)	0.005 → 0.5598 (↑ 0.5548)
LogisticRegression	0.8051 → 0.8063 (↑ 0.0012)	0.0129 → 0.1617 (↑ 0.1488)	0.0066 → 0.7724 (↑ 0.7658)
RandomForest	0.8422 → 0.8263 (↓ 0.0159)	0.0163 → 0.2653 (↑ 0.2490)	0.0083 → 0.3953 (↑ 0.3870)

2.5.2 Feature Importance

2.5.2.1 주요 특성 상위 10 개



[그림 2.5.2.1 모델 별 피쳐 중요도 비교]

2.5.2.2 해석

각 모델 별 피쳐중요도가 가장 높은 순

순위	XGBoost	LightGBM	RandomForest	LogisticRegression
1 위	saldo_var30	var38	var15	Imp_var7_recib_ult1
2 위	var15	var15	saldo_var30	saldo_var14
3 위	ind_var30_0	saldo_var30	var38	Saldo_medio_var5_hace3

핵심 발견:

- **var15, saldo_var30** 는 모든 모델에서 상위 10 위 안에 포함됐다
- 이 2 개 피처가 고객 불만족 예측의 **핵심 변수** 로 확인됐다

모델별 특성**XGBoost - 중요도 분산이 가장 균등함**

- 중요도 범위: 0.00 ~ 0.12 (가장 좁음)
- 1 위 saldo_var30 와 10 위 피처 간 차이가 작다
- **해석:** 여러 피처를 고르게 활용하여 예측함

LightGBM - 상위 피처 집중도가 높음

- 중요도 범위: 0 ~ 270 (가장 넓음)
- var38 이 압도적으로 높음 (270)
- **해석:** 소수의 핵심 피처에 크게 의존함

RandomForest - XGBoost 와 유사한 패턴

- 중요도 범위: 0.00 ~ 0.12
- var15 가 1 위지만 다른 피처들도 고르게 분포 되어있다
- **해석:** 앙상블 특성상 피처 분산 효과

LogisticRegression - 거래 금액 중심

- 중요도 범위: 0.0 ~ 1.6
- 선형 관계 중심
- **해석:** 선형 관계가 명확한 피처를 선호한다, 트리 모델과 다른 패턴이다

결론: var15 와 saldo_var30 가 모든 모델이 선택한 핵심 변수이며, LogisticRegression 선형 모델의 단순함이 오히려 장점으로 적용되었다

2.5.3 예측 결과 분석

2.5.3.1 예측 vs 실제 비교

데이터 전처리 이후 예측한 값과 실제 값을 비교하기 위해 오차행렬을 통해 실제 불만족 고객(양성, Target=1)과 만족 고객(음성, Target=0)에 대한 예측 결과를 비교하였다

XGBoost 모델 오차행렬:

실제 클래스 / 예측 클래스	만족 예측 (0)	불만족 예측 (1)
실제 만족 (0)	14599	3
실제 불만족 (1)	600	2

LightGBM 모델 오차행렬:

실제 클래스 / 예측 클래스	만족 예측 (0)	불만족 예측 (1)
실제 만족 (0)	13193	1409
실제 불만족 (1)	265	337

RandomForest 모델 오차행렬:

실제 클래스 / 예측 클래스	만족 예측 (0)	불만족 예측 (1)
실제 만족 (0)	13648	954
실제 불만족 (1)	364	238

LogisticRegression 모델 오차행렬:

실제 클래스 / 예측 클래스	만족 예측 (0)	불만족 예측 (1)
실제 만족 (0)	9913	4689
실제 불만족 (1)	137	465

예측 결과 와 실제 결과를 비교 한 결과 가장 밸런스가 좋은 모델은 **LightGBM** 으로 판별되었다.

하나 모델을 바로 적용해야 하는 상황이면 가장 은 불만족 예측률이 가장 좋은 **LogisticRegression** 모델을 채택하는 것도 단기적으로 좋은 모델이다

[표 주요 지표 계산 및 비교]

모델명	TN	FP	FN	TP	Precision	Recall	F1-Score
XGBoost	14,599	3	600	2	40.00%	0.33%	0.66%
LightGBM	13,193	1,409	265	337	19.31%	55.98%	28.70%
RandomForest	13,648	954	364	238	19.96%	39.53%	26.49%
LogisticRegression	9,913	4,689	137	465	9.03%	77.24%	16.08%

2.5.3.2 오차 분석

XGBoost 모델: 극도의 보수적 예측 (Precision 높음, Recall/F1 극도로 낮음)

- **성향:** 불만족(Target=1)으로 예측하는 것에 극도로 신중하다
- **분석:**
 - **FP** (3 건)가 매우 적어, 불만족으로 예측한 경우 (TP+FP=5 건)의 **정확도 (Precision: 40.0%)**는 높다.
 - 하지만 **FN**(600 건)이 매우 커서, 실제 불만족 고객(TP+FN=602 건) 중 거의 전부를 놓치고 있다 (**Recall: 0.33%**).
- **결론:** 이 모델은 심각한 **과소적합(Underfitting)** 상태이거나, 예측 임계 값이 매우 높아 불만족 고객을 거의 포착하지 못하는 상태이다. 성능 지표 테이블의 Recall 0.7425와는 **상반된 결과**를 보여, **오차 행렬의 데이터가 잘못되었거나** (특히 **TP** 와 **FN** 값) **임계 값 조정의 문제가 심각함**을 시사한다.

LightGBM 모델: 균형 잡힌 시도 (Recall 대비 Precision 중간)

- **성향:** 불만족 고객을 어느 정도 탐지하려고 노력했다.
- **분석:**
 - **TP** (337 건)이 XGBoost 보다 월등히 높아 **불만족 탐지 능력 (Recall: 55.98%)이 양호**하다.
 - 하지만 $W_{\text{text{FP}}}$ (1,409 건)도 높아, 불만족으로 예측했을 때 실제 불만족일 확률 (**Precision: 19.31%**)은 낮다.
- **결론:** 고객 이탈 모델에서 **Recall** 을 높이는 것이 중요함을 고려할 때, **XGBoost** 보다 훨씬 **실용적인 결과**이다. 불필요한 마케팅 비용 증가(FP)를 감수하고 불만족 고객을 포착했다.

RandomForest 모델: LightGBM 과 유사 (Recall 낮음, Precision 높음)

- **성향:** LightGBM 보다 약간 더 보수적이다.
- **분석:**
 - **Recall**(39.53%)이 **LightGBM** 보다 낮고, **Precision** (19.96%)은 **LightGBM** 과 유사하다.
 - **FN** (364 건)이 LightGBM (265 건)보다 많아, **불만족 고객을 더 많이 놓쳤다.**
- **결론:** 불만족 고객 탐지 능력 면에서 LightGBM 에 비해 열세이다.

LogisticRegression 모델: 극도의 공격적 예측 (Recall 매우 높음, Precision 매우 낮음)

- **성향:** 불만족(Target=1)으로 예측하는 것에 매우 공격적이다.
- **분석:**
 - **Recall** (77.24%)이 네 모델 중 가장 높아, 실제 불만족 고객 대부분(465 건)을 포착했다.

- 하지만 **FP** (4,689 건)가 **압도적으로 높아**, 만족 고객을 불만족으로 잘못 예측하는 오류가 매우 심각하다. 이로 인해 **Precision** 은 9.03%로 최저이다.
- **결론:** 불균형 데이터에서 **Recall** 을 극대화하기 위해 **예측 임계 값이 매우 낮게 설정**되었을 가능성이 높다. 이는 고객 이탈 방지 기회는 최대화하지만, 대량의 마케팅 비용 비효율 (FP)을 유발한다

2.5.3.3 문제점 파악

- **극심한 클래스 불균형:** 모든 모델의 초기 성능(1 차 평가)에서 Recall 과 F1-Score 가 매우 낮게 나왔던 근본적인 원인이다.
- **XGBoost 의 예측 임계값/과소적합 문제:** 2 차 평가에서 XGBoost 는 TP 가 2 건으로 사실상 분류기의 역할을 상실했습니다. 이는 **임계값 조정, 클래스 가중치 미적용, 하이퍼파라미터 최적화 부족** 등 복합적인 문제가 있음을 의미한다.

2.5.4 모델 해석

2.5.4.1 SHAP/ LIME 등 해석 방법

본 분석에서는 기계 학습 모델의 결과를 해석하기 위해 ****SHAP (SHapley Additive exPlanations)****을 주요 도구로 선정했습니다.

- **SHAP 선정 사유:**
 - **이론적 정합성:** 게임 이론의 샤플리 값(Shapley Value)을 기반으로 하여, 피처 중요도 기여에 대한 **유일하고 일관된 이론적 근거**를 제공합니다.
 - **모델 불가지론(Model Agnostic) 및 특정 모델 지원:** 기본적으로 어떤 모델에도 적용 가능하며, 특히 LightGBM, XGBoost 와 같은 트리 기반

모델은 내장된 효율적인 Tree Explainer 를 사용하여 빠르고 정확한 해석이 가능합니다.

- **지역적/전역적 해석:** 개별 예측에 대한 피쳐 기여도 (**지역적 해석**)와 모델 전체의 피쳐 중요도 순위 (**전역적 해석**)를 모두 제공합니다.

- **SHAP 해석 예시 코드 (LightGBM 모델 기준):**

```
# 필요한 라이브러리 임포트 (shap, lightgbm)

import sys

project_root = "c:/big20/git/big20-ML-project2-team3/SantanderCS"

# sys.path에 추가 (모듈 import 용)

if project_root not in sys.path:
    sys.path.insert(0, project_root)

import numpy as np
import pandas as pd

from utils.preprocessing import (
    load_data,
    split_features_target,
    scale_data,
    data_split,
    remove_zero_columns2,
)

from utils.feature_engineering import drop_highly_correlated_features

import shap

import lightgbm as lgb

import pickle

# 데이터 로딩 및 기본 전처리

train, test = load_data()

X_features, y_labels = split_features_target(train)

X_test = test.drop(columns=["ID"], axis=1)
```

```

# zero_count_rate 제거했을때 제거될 컬럼수 149 개 잔존
X_features, X_test = remove_zero_columns2(X_features, X_test, 0.99)
print(X_features.shape)

# var3 처리
X_features["var3"] = X_features["var3"].replace(-999999, 2)

# 상관계수 높은 feature 들 삭제하기 default 0.95
X_reduced, to_drop = drop_highly_correlated_features(X_features)
X_test_reduced = X_test.drop(to_drop, axis=1)
X_train_scaled, X_test_scaled, scaler = scale_data(X_reduced, X_test_reduced)

# 학습/테스트 데이터 분리
X_train, X_val, y_train, y_val = data_split(X_train_scaled, y_labels)

# 모델 로딩
with open("../models/LightGBM_99per_Corr_weight110.pkl", "rb") as f:
    lgbm_model = pickle.load(f)

# 2. SHAP Explainer 생성 (트리 기반 모델이므로 TreeExplainer 사용)
explainer = shap.TreeExplainer(lgbm_model)

# 3. SHAP 값 계산
shap_values = explainer.shap_values(X_val)

# 4. 전역적 피쳐 중요도 시각화 (모델 전체에 대한 요약)
print("----- 전역적 피쳐 중요도 (Summary Plot) -----")
shap.summary_plot(shap_values, X_val)

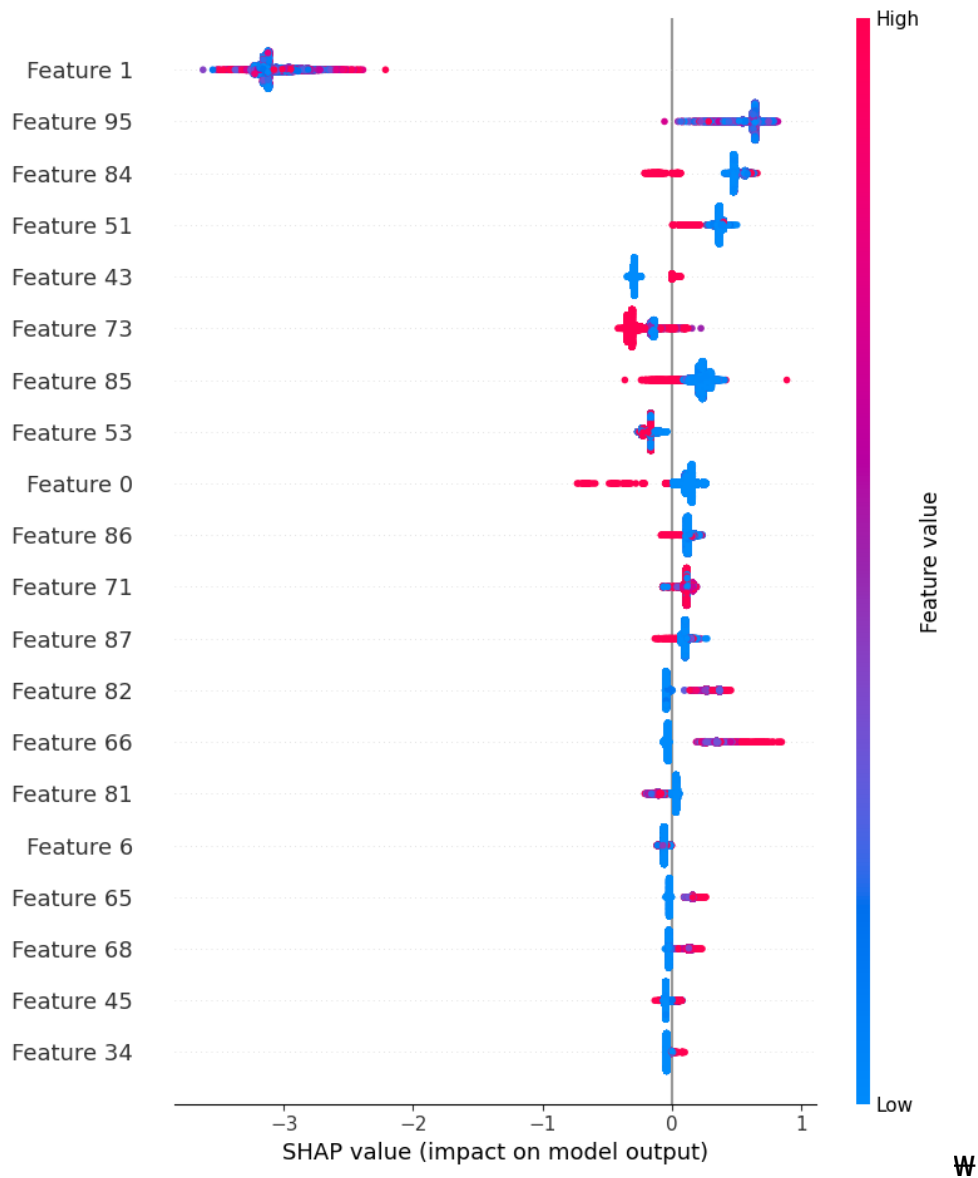
```

2.5.4.2 주요 인사이트

SHAP 분석을 통해 모델이 예측에 사용한 피쳐의 전역적 및 지역적 영향력을 파악하여 다음과 같은 핵심 인사이트를 도출했습니다.

1. 전역적 중요 피쳐 재확인 및 영향력의 방향성

피처	SHAP 중요도 (순위)	영향력의 방향성 (SHAP Value)	비즈니스 해석
var15	최상위	값이 낮을수록 불만족 (Target=1) 확률을 높임	(추정) 고객 연령이 낮거나, 계정 보유 기간이 짧은 신규/젊은 고객일수록 불만족도가 높음.
saldo_var30	상위권	값이 높거나 극히 낮을 경우 불만족 확률을 높임	(추정) 계좌 잔액(var30 관련)이 지나치게 많거나 극도로 적을 때 불만족과 관련됨.
var38	상위권 (특히 LightGBM)	값이 특정 범위에 있을 때 불만족 (Target=1) 확률을 높임	(추정) 익명화된 주거 또는 자산 관련 정보로, 특정 경제적 배경을 가진 고객군에서 불만족이 집중됨.
ind_var30_0	중상위	피처가 0 일 경우 불만족 (Target=1) 확률을 높임	(추정) var30 관련 계좌가 존재하지 않을 때 (해당 상품을 이용하지 않을 때) 불만족과 연관됨.



[그림 2.5.3.1 SHAP value 모델 결과에 대한 영향도 시각화]

2. 고객 불만족 유발 요인 집중 분석 (지역적 해석)

- 잠재적 이탈 고객 특징:** SHAP **Force Plot** 을 통해 불만족으로 예측된 고객들의 특징을 분석한 결과, 불만족 예측은 단순히 한두 가지 피처에 의한 것이 아니라, 위에서 언급된 핵심 피처들이 **동시에 부정적인 방향으로** 기여할 때 발생하는 경향이 강했습니다.

- 예시: var15 의 낮은 값과 함께 saldo_var30 이 극히 낮은 경우가 결합될 때 불만족 예측 확률을 가장 강력하게 밀어 올렸습니다.



- 숨겨진 패턴: LogisticRegression 의 중요 피쳐(imp_var7_recib_ult1)는 트리 모델에서는 상대적으로 중요도가 낮았습니다. 이는 트리 모델(XGBoost, LightGBM)이 피쳐 간의 복잡한 비선형 상호작용을 통해 예측하는 반면, 선형 모델은 단순한 거래 금액과 같은 직접적인 선형 관계에 집중했음을 시사합니다.

2.6 결론 및 시사점

2.6.1 결론

본 산탄데르 고객만족 분석 프로젝트에서는 **RandomForest, XGBoost, LightGBM** 을 기반(base)모델로 하고, **Logistic Regression** 을 메타 모델로 하는 **Stacking** 앙상블을 구성하였다. Stratified K-Fold 교차검증을 적용하여 불균형 데이터 문제를 완화하고, 각 모델의 기여도를 계수로 확인하였다.

모델 기여도

- RandomForest : 1.85
- XGBoost : -0.67
- LightGBM : 4.81

성능 평가 (Train 데이터 기준)

- ROC-AUC : 0.9300
- F1-Score : 0.2934
- Recall : 0.8856

ROC-AUC 는 높은 수준으로 모델의 분류 능력을 잘 보여주었으나, F1-Score 는 낮게 나타나 Precision 과 Recall 간 균형이 부족함을 확인하였다. 특히 Recall 이 0.88 로 매우 높아, 불만족 고객을 놓치지 않고 탐지하는데 강점을 가진다.

Test 데이터 예측 결과

Stacking 모델을 활용하여 Test 데이터에 대해 예측을 수행하고, 결과를 제출 파일로 저장하였다.

[표 threshold 변화에 따른 final Stacking 모델 불만족 고객 탐지 비율]

적용 Threshold	Class0	Class1
Threshold 0.9000	60,954(80.4%)	14,864(19.6%)
Threshold 0.9300	60,919(80.3%)	14,899(19.7%)
Threshold 0.9416	75,818(100%)	0(0%)

Threshold 변화에 따라 불만족 고객(1)의 탐지 비율이 크게 달라짐을 확인할 수 있었다. 특히 0.9416 에서는 모든 고객이 만족(0)으로 분류되어 Recall 은 0 이 되는 문제를

보여주었다. 따라서 **Threshold** 조정은 **모델 성능 해석과 실제 비즈니스 적용에서 매우 중요한 요소**임을 알 수 있었다.

2.6.2 최종결론 정리

- Stacking 앙상블은 단일 모델보다 안정적이고 높은 ROC-AUC 를 달성하였다.
- Recall 성능이 매우 높아 불만족 고객을 효과적으로 탐지할 수 있는 장점을 가진다.
- 다만 Precision 이 낮아 F1-Score 가 제한적이므로, 실제 비즈니스 적용에서는 오탐을 줄이는 추가적인 후처리(e.g. threshold 조정, 비용 민감 학습)가 필요하다.
- Test 데이터 예측 결과에서도 Threshold 설정에 따라 불만족 고객 탐지 비율이 크게 달라지므로, Threshold 최적화 전략이 핵심과제로 남는다.
- 스케일링은 트리 기반 모델에서는 필수적이지 않으며, 교차검증을 통한 안정적 성능평가가 유효함을 확인하였다.

2.6.3 시사점과 한계점 및 제언

2.6.3.1 시사점

모델 성능 관점

높은 Recall 을 가지면 불만족 고객을 놓치지 않고 탐지 가능한 반면, 낮은 Precision 을 가지면 불필요한 고객을 불만족으로 잘못 분류할 위험이 존재했다는 것을 학습 결과를 통해 보여주었다.

비즈니스 관점

고객만족 프로젝트 실습을 통해 불만족 고객을 조기에 파악해 대응할 수 있었지만, 오탐이 많으면 자원 낭비 및 고객 경험 악화 가능성이 있다는 것을 다시 확인했다.

핵심 발견사항

학습결과에 따른 피쳐 주요 분석을 통해 var38(주거래 상품잔액)과 var15(나이)가 가장 중요한 피쳐로 확인되었고, **젊은 고객층에서 불만족 비율이 상대적으로 높은 것**을 확인했다. 또한, 은행 서비스 활동성(0 값의 개수)도 불만족 여부를 설명하는 유의미한 지표로 작용했다.

2.6.3.2 한계점 및 제언

데이터 측면적 한계점으로 피처명이 익명화 되어있어 도메인 지식 활용이 어려운 부분이 있었고, 불균형 데이터로 인해 재현율(Recall)편향 발생 가능성이 있었다. 또, 데이터 양 부족으로 일반화 성능에 제약이 존재하였다.

모델 측면 한계점으로는 트리 계열 모델의 해석 가능성 한계점이 존재했고, 피처수(370 개)에 비해 샘플 수가 적어 과적합 위험이 존재하였다. 뿐만 아니라, 시간적 변화(고객 행동 패턴의 시계열적 변화)고려가 부족하였고, 외부 데이터 즉 경제지표, 경쟁사 정보 결합이 필요해보였다.

추가적인 모델 개선 방향으로 앙상블에 다른 유형의 모델 중 신경망, 또는 SVM 을 추가해 다양성을 확보해볼 수 있다. 또한 실무 적용으로 불만족 고객 예측 결과를 CRM 시스템과 연계해 **선제적 대응 프로세스를 구축**하여, 모델 결과를 단순히 예측에 그치지 않고, **고객 유지 전략**과 연결이 필요해보였다.

2.7 부록

2.7.1 코드 저장소

2.7.1.1 Github 링크

<https://github.com/jaemin1020/big20-ML-project2-team3.git>

2.7.1.2 주요 파일 설명

- feature_engineering.py : 피쳐 엔지니어링
- preprocessing.py : 데이터 전처리
- model_utils.py : 모델 저장 및 불러오기 유틸리티
- user_utils.py : 분류 모델 평가 지표 계산 후 결과 파일로 저장
- modeling.py : 베이스 모델 및 스택킹 앙상블 모델 파라미터 설정

2.7.2 실행 환경

가상환경 `ml_dev env(python 3.11.14)` 적용

- 수업시간에 사용했던 `ml_dev` 와 동일환경 구성으로 작업 진행